DJing in Latent Space: Designing a MIDI DJ Controller for Deep Learning Interaction on the Disklavier

Ivana Rasch Chinchilla

Submitted in partial fulfillment of the requirements for the
Master of Music in Music Technology
in the Department of Music and Performing Arts Professions
Steinhardt School
New York University

Advisor: Dr. Magdalena Fuentes
Reader: Dr. Agniezka Roginska
May 2, 2026

Abstract

This thesis presents the design, implementation, and evaluation of a custom MIDI DJ controller for live interaction with MusicVAE, a variational autoencoder that encodes MIDI melodies into a latent space. The system enables a performer to interpolate between two musical sequences using a touchbar, manipulate note density and melodic interval through knobs, adjust tempo via a contactless distance sensor, scrub through melodies via platters equipped with magnetic angle sensors, with all output routed in real time to a Yamaha Disklavier acoustic piano. The controller was built using open-source hardware and software tools, including a Teensy 4.1 microcontroller, a custom PCB, and a laser-cut recycled material enclosure. Four audio-domain deep learning models were explored prior to adopting MusicVAE, each failing to satisfy the requirements of content preservation and real-time controllability. Quantitative evaluation showed that interpolation produced smooth, monotonically consistent melodic transitions across all tested song pairs, and the note density attribute vector achieved a mean Spearman rank correlation of $\rho = 0.989$. The average melodic interval vector produced weaker results ($\rho = 0.362$), likely reflecting entanglement with pitch-related attributes in MusicVAE's learned representation. These results suggest that latent space DJing is a technically viable and musically promising performance paradigm, and that the expressivity of the system ultimately depends on whether the model has learned to organize musical attributes in a way that physical controls can address.

Acknowledgements

I would like to thank my NYU professors, especially my advisor Dr. Magdalena Fuentes, for opening new directions; Professor Mason Mann, whose work in computer music and DIY instrument building directly inspired this thesis; and Dr. Agnieska Roginska, who pushed me hard at the start and became one of my greatest sources of reassurance when I needed it most. I would also like to acknowledge my undergraduate mentor, Dr. William Brent, who first introduced me to digital musical instruments and machine learning.

I am profoundly grateful to my parents for supporting this degree without ever fully understanding its purpose, which honestly might be the purest form of support and the reason I ended up here at all. I would especially like to thank my mom for making me take piano lessons after watching The Pianist and deciding that, just in case, I should have a way to not starve. I don't know that any of this would exist without those lessons.

Though he is currently testing my patience, I want to thank my friend Guillermo for showing me what it means to love music and making those forced piano lessons feel like something I chose. I also wouldn't have been able to complete this work without my best friend Andrea and my NYC friends for putting up with what can only be described as a sustained and emotional thesis monologue. I would especially like to thank Georgia, and Maria Magdalena for always making difficult days better, and Mary and Ramsey, whose conversations about music, technology, and ethics inspire my work to engage with technology both critically and creatively.

This thesis is also dedicated to my siblings, who I know will become great engineers and scientists someday. Being their older sister inspires me to show them the most beautiful parts of these fields in everything I do.

Above all, I want to thank every woman and minority who pushed open doors that I got to walk through. I am aware that my being here is not only the result of my own work, but of every person who had to fight harder and prove more just to be taken seriously in these fields. I do not take that lightly, and I never will.

Contents

List of Figures

List of Tables

1 Introduction

Debates about authenticity in music have always followed the introduction of new technology. Instruments such as the electric guitar, the synthesizer, and the drum machine were all met with skepticism when first introduced, yet these were the very instruments that gave birth to genres many people today cannot get enough of. The turntable, similarly, dismissed as illegitimate, became the foundation of an entire musical culture. Unlike those instruments, which eventually shed their controversy, the DJ controller remains a site of ongoing criticism, and sometimes rightly so. While it has elevated the practice of curation as a musical art, it has also made it possible to press play on a pre-recorded track and call it a performance. In response to these tensions, the rise of deep learning and the increasing accessibility of hardware and open-source tools have created new possibilities for building personalized instruments that go beyond what commercial controllers have to offer. This thesis takes the position that engaging critically with the tools of the present moment is not a departure from musical authenticity but a continuation of it.

Across the history of DJ technology, new tools have consistently been designed to feel like natural extensions of what came before. When vinyl gave way to CDs, manufacturers placed a jog wheel on the CDJ so DJs could still spin and scratch. When music went digital, the software kept the two-deck layout and the crossfader. [Rietveld, 2016] calls this skeuomorphism: designing new technology to look and feel like what came before. Previous attempts to apply deep learning to interactive performance systems, particularly those involving manipulation of latent space, often struggled precisely because they abandoned this principle by building entirely new physical forms that gave performers no familiar gestural foothold. This thesis takes the opposite approach, borrowing the established gestures of DJing to make new kind of musical material legible. This matters because, as [Meyer, 1956] argues, musical meaning arises from the tension between expectation and surprise. Familiar gestures carry familiar expectations, while unfamiliar ones invite exploration and a disruption of established practice. This logic extends to the musical output of the system as well. Rather than generating music with no point of reference, the interpolation departs from two known melodies and returns to them, so the generated material in between is explorative but never completely unmoored, mirroring the way DJ controllers have always worked.

The output of the system adds a final layer to this logic. The signal travels from an abstract mathematical space through a MIDI cable into a Yamaha Disklavier, a self-playing acoustic piano that strikes real strings with real hammers in real

time. The most intangible kind of musical material is realized through one of the most physical kinds of sound production. It is within this context, where new technology is embedded in familiar instruments and building something by hand is part of the creative process, that this thesis asks the following questions:

1. What are the technical and ergonomic limitations of using physical controllers to navigate MusicVAE latent spaces?

2. What does the process of building a custom DMI for deep learning interaction reveal about the current feasibility of this kind of system for individual instrument builders working with open-source tools?

3. What are the limitations of current audio-domain and MIDI-domain deep learning music models for controllable and content-preserving real-time performance?

4. How can latent space interpolation and attribute vectors in MusicVAE function as control parameters for musical features such as density, contour, and key, and where do these controls break?

2 Prior Work

The development of this project sits at the intersection of digital musical instruments (DMIs) and neural generative models. It extends traditional DMI research by integrating latent-space music generation into a physically embodied controller which enables real-time manipulation of melodic sequences. Rather than focusing solely on interface design, this research investigates how deep learning models can be embedded within performative systems to enable new forms of musical control and expression. The following literature situates this work within existing research on DMIs, neural music generation, latent space models, latent space manipulation interactive systems, and acoustic-digital hybrid instruments.

2.1 Digital Musical Instruments

2.1.1 Embodied Interaction in DMIs

The proliferation of music creation technologies has largely been delivered through general-purpose interfaces designed for spreadsheet's and writing emails, where interaction is reduced to mouse clicks and keyboard input. [Leman and Godøy, 2010] argue that music is inherently multimodal and that gesture is not peripheral to musical experience but central to it. [Leman, 2007] extends this by proposing that the human body mediates between physical sound energy and musical meaning concluding that interaction is a condition for meaningful musical engagement. If musical understanding is grounded in physical gesture, interfaces that reduce performance to clicking and dragging numbers on a screen may risk the connection between bodily movement and musical expression. The system developed in this thesis builds on this premise by designing a DJ controller that brings MusicVAE's latent space interpolation beyond a browser-based demonstration into a physically embodied performance interface. DJing offers a concrete case study in how embodied gesture and music technology have co-evolved, and how digital transitions have put that relationship under pressure.

2.1.2 DJ Controllers

The term "disc jockey" emerged in 1940s radio broadcasting, where playing recorded music in place of live performance was initially criticized by musicians and radio professionals [Katz, 2012]. DJing has since evolved substantially into the contemporary digital landscape of software and controllers. [Montano, 2010] notes

that while this shift has democratized DJing and expanded creative possibilities, it has also unsettled definitions of authentic DJ skill, with many in the dance music community remaining skeptical of performances that no longer involve vinyl or turntables. The DIY ethos explored by [Northlich, 2013] represents a response to this tension by encouraging musicians to build custom interfaces that restore a more intimate, hands-on relationship with the music. This thesis extends that tradition by designing the custom controller to reimagine its purposes.

[Vieira and Schiavoni, 2020] demonstrate an example of an affordable Arduino-based DIY MIDI controller designed for DJ performance. This low-cost DIY approach to building a MIDI controller addresses the socioeconomic barriers to accessing music technology, particularly in underdeveloped regions. By using open-source technologies and inexpensive components, DIY methodologies foster digital inclusion and expand musical opportunities for underserved communities. This ethos of accessibility is also explored in [Northlich, 2013] thesis on experimental electronic music which addresses how the DIY movement encourages artistic freedom and the repurposing of technology to create unique sounds.

2.1.3 Deep Learning in DJ Controllers

Deep learning is starting to reshape what DJ controllers can do; for example, [Sontag et al., 2024] present No-IDLE, a prototype system that incorporates real-time user feedback such as speech and gaze tracking that allow controllers to adapt dynamically to a DJ's performance. [Rydén, 2026] takes a closer step by evaluating latent space interpolation as a DJ transition operator. Their methodology consists of encoding bar-aligned audio segments with EnCodec and blending them via spherical linear interpolation. The results indicate that while latent space interpolation is a viable approach to DJ transitions, achieving perceptual quality comparable to conventional crossfades remains an open challenge. Despite operating in the audio domain, this work directly motivates the approach taken in this thesis, which applies the same principle of latent space interpolation to the symbolic MIDI domain using MusicVAE.

2.1.4 Hybrid Acoustic - Digital Instruments: The Disklavier

The Disklavier, a self-playing piano, has been used as a platform for combining acoustic performance with live algorithmic control. [Dahlstedt, 2014] demonstrates this in "Circle Squared" and "Circle Keys," where generative algorithms interact with the instrument through pressure sensors and direct piano input. This

highlights how algorithmic systems can enable new forms of expressive control over musical structure. [Fiordelmondo et al., 2023] extend this further in ”Il Caos delle Sfere,” where the Disklavier responds to the actions of a pinball machine. Together these works establish the Disklavier as an instrument with a history of algorithmic control; this thesis extends that tradition by evolving from rule-based algorithms toward a neural generative model.

2.2 Deep Learning for Music Generation

In DJ performance, transitioning smoothly between songs is central to the practice. This makes style transfer a particularly relevant framework for this work, as it offers a principled way of thinking about how musical characteristics can be carried across from one source to another. This subsection examines the areas of style transfer and generative music models that have potential application to DJing. This is approached by tracing the evolution from image-based style transfer to its musical extensions and ultimately to generative models operating in the MIDI domain.

2.2.1 Neural Style Transfer

Neural Style Transfer (NST) was first introduced by [Gatys et al., 2016], as an approach to combine the content of one image with the artistic style of another using convolutional neural networks (CNNs). The technique emerged from the broader field of deep learning, which was focused on using CNNs for tasks like image classification. [Gatys et al., 2016] discovered that CNNs could extract and separate high-level content and style representations from images, enabling the transformation of images to mimic the aesthetic qualities of famous paintings like Van Gogh and Edvard Munch while preserving their underlying structure. Since then, NST has evolved to apply not only to images but also to other domains like music, expanding the possibilities for creative applications in real-time performance and digital art.

Before diving deeper into specific approaches it is worth acknowledging that style is not a scientifically settled concept. [Dai et al., 2018] argue that depending on how researchers interpret musical style, studies nominally addressing the same problem are often solving fundamentally different ones. They propose distinguishing between timbre style transfer, performance style transfer, and composition style transfer as distinct sub-problems, each corresponding to a different level of music representation. This taxonomy is useful for situating the approaches

surveyed below: audio-domain methods tend to operate at the timbre level, while the symbolic MIDI approach operates closer to the composition level. This thesis explores both, though compositional style transfer is the approach adopted in the final system.

2.2.2 Symbolic and Audio Representation in Music Generation

Beyond visual art, NST has been adapted for music, most notably through spectrogram-based approaches. [Wyse, 2017] explored applying CNNs to audio spectrograms, noting the challenges that arise from treating them as images due to fundamental differences in how visual and audio information is structured. More recently, [Javadi, 2026] demonstrated timbre transfer within the pretrained latent space of Descript Audio Codec, finding that latent space transformations produced novel creative effects but fell short of conventional tools in perceptual realism. [Team et al., 2025] extends this landscape by enabling continuous audio generation conditioned on text or audio prompts, though their fundamentally generative architecture means they produce new content rather than transforming existing material. [Engel et al., 2020] takes a contrasting approach, representing audio through harmonic and noise components to provide direct control over pitch and timbre. VAE and diffusion-based timbre transfer models such as [Caillon and Esling, 2021] and [Demerlé et al., 2024] similarly pursue content-preserving audio transformation but remain constrained by artifacts and limited generalization.

In the MIDI domain, style transfer is approached differently. [Yang et al., 2017] generates harmonically coherent MIDI sequences through GAN-based conditioning, however, GANs struggle with the long-term dependencies essential for expressive music. [Brunner et al., 2018] addresses this by encoding musical sequences into a latent space, enabling more flexible control over dynamics and articulation. [Cífka et al., 2020] advances this further with an end-to-end supervised framework for accompaniment style transfer, with generalization to styles outside its training distribution remaining an open challenge.

2.3 Latent Space Models and Real-Time Interaction

The models most relevant to this thesis operate by encoding musical material into a continuous latent space that can be navigated and manipulated in real time. Variational Autoencoders (VAEs) were first introduced by [Kingma and Welling, 2013]. Their functionality is to compress input data into a lower-dimensional

latent space represented as probability distributions which enable smooth interpolation between encoded points, a property particularly well suited for live performance contexts. The following subsections examine how latent space representation, interpolation, and attribute-based control have been applied in generative music systems and how these techniques inform the system developed in this thesis.

2.3.1 Latent Space Representation and Interpolation

[Roberts et al., 2018] is a hierarchical variational autoencoder developed by Google Magenta that learns a continuous latent space of musical sequences and enables smooth interpolation between two encoded melodies. The model uses a hierarchical LSTM decoder in which a conductor RNN first generates bar-level embedding vectors that initialize a lower-level decoder LSTM, which then autoregressively generates individual notes for each subsequence. Interpolation between two encoded sequences is performed via spherical linear interpolation in the latent space proposed by [White, 2016]. Spherical linear interpolation is chosen to avoid the low-density regions that straight linear paths tend to pass through in high-dimensional Gaussian spaces. The result is a series of intermediate sequences that transition smoothly and musically between two melodic endpoints which maps directly onto the DJ performance context where movement between two songs is crucial.

2.3.2 Attribute Vectors and Semantic Control

As generative music models have advanced, so too has interest in making their latent spaces more meaningful and accessible to performers and composers. The question of how users can meaningfully interact with and control generative music models has driven significant research into latent space interpretability. Roberts et al. [Roberts et al., 2018] demonstrated that MusicVAE's latent space supports attribute vector arithmetic. This involves computing a direction in latent space by averaging encoded examples that share a given attribute, then adding or subtracting that vector to manipulate the corresponding musical quality in the decoded output. While effective, this approach does not guarantee that attributes align with specific latent dimensions, since the model is not explicitly trained to enforce this structure. Pati et al. [Pati and Lerch, 2019] address this limitation by proposing a regularization technique that encodes selected musical attributes along specific dimensions of a hierarchical VAE's latent space, demonstrating that traversing a

regularized dimension produces outputs with monotonically increasing attribute values, far exceeding an unregularized baseline for attributes including rhythmic complexity and pitch range.

Kawai et al. [Kawai et al., 2020] take a complementary approach, conditioning the decoder on continuous attribute vectors while using an adversarial classifier-discriminator to force the encoder to remove attribute information from the latent vector entirely. This methodology enables explicit and independent control over multiple continuous attributes simultaneously. More recently, Sharma et al. [Sharma and Baghel, 2026] extend this direction by architecturally separating pitch and rhythm into distinct latent subspaces within a disentangled VAE, demonstrating that independent traversal of each subspace produces the expected musical changes without cross-contamination between attributes. The system developed in this thesis adopts the attribute vector method from Roberts et al., computing density vectors directly from the frozen pretrained MusicVAE [Roberts et al., 2018] without retraining.

2.3.3 Latent Space Manipulation for Interactive Systems

Latent space navigation has been explored as a form of embodied musical performance in research presented at the New Interfaces for Musical Expression (NIME) conference, where generative models are embedded within interactive systems for real-time performer engagement. [Scurto and Postel, 2023] map physical movement through a three-dimensional environment directly to a RAVE latent space, concluding that such interfaces can foster new AI practices that do not encroach on existing musical labor, positioning the performer as actively shaping the result rather than a passive recipient of automated output. [Privato et al., 2024] also use physical gesture to navigate RAVE, embedding the latent space within a DMI where performers displace magnetic spheres across a board. This might seem to contradict the control-oriented design of this thesis, but the preference for instability likely reflects the abstract nature of the interface itself. When there is no clear relationship between a physical action and its musical result, exploration becomes the only available mode of engagement. The DJ controller developed in this thesis takes the opposite approach, grounding latent space navigation in familiar gestures whose musical outcomes are predictable and intentional.

Other works take a more deliberate approach to latent space control. [Shaheed and Wang, 2024] embed performers audio within a RAVE feedback loop in *I Am Sitting in a (Latent) Room*, where individual latent dimensions are manipulated directly through addition, multiplication, and muting. They also treat dataset

construction as part of the compositional process itself, which demonstrates how meaningful human intervention can be built into every layer of a generative system. [Warren and Çamci, 2022] present *Latent Drummer*, a system that uses a VAE and Markov models to generate drum patterns from simple drawings. Taking a complementary approach, [Murray-Browne and Tigas, 2021] propose that the mapping between gesture and latent space can itself be a creative act, training a VAE on the creator's own gestural recordings so that the latent space structure emerges from their movement.

The system developed in this thesis occupies a specific position within this literature. Rather than treating latent space as an open territory to be explored freely, it embeds MusicVAE latent space navigation within the established affordances of a DJ controller, grounding interaction in the logic of live performance practice. The crossfader maps directly onto the interpolation path between two encoded melodies and dedicated knobs apply attribute vectors for density and interval directly within the latent space. The works reviewed here demonstrate that latent space navigation is a promising and active area of musical performance research. This thesis builds on that foundation, bringing the same creative ambition to a more constrained and familiar physical form.

3 Method

3.1 Research Approach: Practiced Based Research

This thesis adopts a practice-based research (PBR) methodology, in which knowledge is gathered through the iterative design, construction, and evaluation of a functional system. As outlined in the UK Arts and Humanities Research Council's Practice-Based Research Guide [Candy, 2006], practice-based research produces new understanding through creative practice itself, where the processes of making, testing, and reflection constitute valid forms of research knowledge. Within this framework, creative practice functions as both the method and the site of inquiry.

The research is broken down across three interconnected sites of practice. The first is the design and fabrication of the custom DIY DJ controller, developed across two breadboard prototypes and a final PCB. Each iteration exposed new hardware constraints that shaped subsequent decisions. The second is the selection and integration of a generative model, which involved systematic experimentation across both audio and MIDI domain approaches before arriving at MusicVAE as the final implementation. Each model explored was selected in response to the limitations of the previous one. The third is the development of a browser-based software interface, built iteratively from an existing open-source foundation and extended to support real-time MIDI performance. This included operating directly within MusicVAE's latent space for crossfader interpolation between encoded melodies and attribute vector control over musical properties. Throughout this process, design decisions are treated as experimental variables refined across successive iterations.

3.2 System Overview

This research presents a custom DIY DJ controller and a browser-based MusicVAE interface that together form a system for real-time neural melody interpolation. A performer uses the physical controller to navigate the latent space between two input melodies, and the system outputs the interpolated MIDI sequences to a Yamaha Disklavier acoustic piano.[1]

The system has three main components. The controller is built around a Teensy microcontroller and includes a touch bar, magnetic rotary encoders, capacitive touch, potentiometers, buttons, and time-of-flight distance sensors. It sends

[1]The full source code for the system is available at `https://github.com/ivanarasch/Ivanita-s-DIY-DJ-Controller.git`.

MIDI control data to the browser interface via the Web MIDI API. The interface runs MusicVAE locally and computes an 11-step interpolation between two input melodies in latent space, exposing a set of real-time performance parameters including pitch shifting, octave, note length, swing, looping and navigation controls. Two of these parameters, density and interval, control the generative process itself. As the performer moves through the parameters, the corresponding MIDI sequences are sent to the Disklavier for acoustic playback.

3.3 Hardware Design

The physical controller is a custom-built DIY DJ controller centered around a Teensy 4.1 microcontroller.[2] It integrates five categories of sensing modalities. Fourteen potentiometers, comprising twelve knobs and two sliders, provide continuous rotary and linear control. Twenty tactile buttons provide discrete input for triggering events. A Trill Bar capacitive touch sensor[3] allows for sliding gesture and finger position detection. Two AS5600 magnetic rotary encoders[4] are mounted beneath custom 3D-printed platters and detect rotational position and direction. Two VL53L0X time-of-flight distance sensors[5] enable non-contact gestural interaction by measuring the distance of the performer's hand above the sensor surface.

The four I2C devices, comprising of two AS5600 encoders, two VL53L0X sensors, and Trill Bar, are distributed across two I2C buses. This was necessary because both the AS5600 and the VL53L0X have fixed I2C addresses that cannot be changed in software, making it challenging to have two devices of the same type on the same bus without address conflict. Separating them across two buses ensures unambiguous communication between the Teensy and all sensors. All other controls connect directly to the Teensy's analog and digital GPIO pins. The end goal is for all sensors and controls to communicate with the Teensy, which outputs MIDI messages to enable communication with the browser-based interface.

[2]The Teensy 4.1 is a microcontroller development board produced by PJRC. `https://www.pjrc.com/store/teensy41.html`

[3]The Trill Bar is a capacitive touch sensor produced by Bela. `https://shop.bela.io/products/trill-bar`

[4]Adafruit AS5600 magnetic rotary encoder breakout board. `https://www.adafruit.com/product/6357`

[5]Adafruit VL53L0X time-of-flight distance sensor breakout board. `https://www.adafruit.com/product/3317`

3.3.1 Fabrication and Enclosure

The controller was developed through two breadboard prototypes before moving to a PCB and final enclosure. The first prototype consisted of six knobs and four buttons wired to the Teensy, with the goal of establishing basic MIDI communication. Mixxx[6] was used as a validation environment because its MIDI Learning Wizard made it straightforward to confirm that the Teensy was transmitting correctly and functioning as a DJ controller.

The second breadboard prototype expanded the system to eight knobs, eight potentiometers, two AS5600 encoders, one VL53L0X distance sensor, and the Trill Bar, with all sensor types working together for the first time, as shown in Figure 1. Two firmware decisions were made during this phase that shaped the final implementation. The buttons were originally wired to ground with external pull-down resistors, but switching to the Teensy's internal pull-up resistors using the `StevesAwesomeButton` library[7] eliminated the extra wiring and simplified the layout considerably. The `ResponsiveAnalogRead` library[8] was also introduced at this stage to address noise in the potentiometer readings, which were producing unstable MIDI output even when the knobs were not being touched.

Once the breadboard prototype was stable, the PCB was designed in KiCad[9] following the schematic developed during the prototyping phase (see Appendix A and Figure 2). Laying out the board required routing connections for fourteen potentiometers, twenty buttons, two I2C buses, and five sensors simultaneously, which made trace routing and avoiding crossovers a significant challenge. This was the first PCB designed for this project, and some of the complexity of the layout contributed to the footprint errors discovered after fabrication.

[6] Mixxx is a free and open-source DJ software application. `https://mixxx.org`

[7] `https://github.com/crudlabs/StevesAwesomeButton`

[8] `https://github.com/dxinteractive/ResponsiveAnalogRead`

[9] KiCad is a free and open-source electronics design automation suite. `https://www.kicad.org`

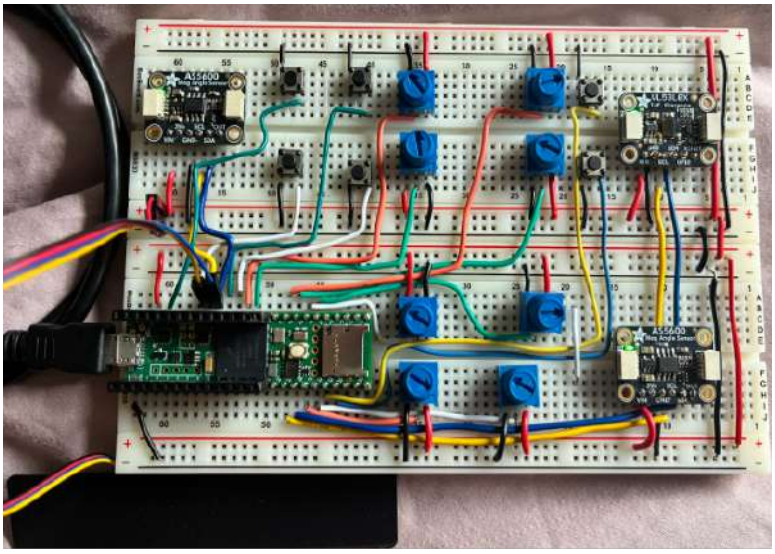

Figure 1: Second Breadboard Prototype.

When the board arrived, several errors were found. On the left deck, the pin headers for the AS5600 encoder were placed mirrored, so the pin order was backwards and the component could not be mounted directly. On the right deck, the footprints for the distance sensor and the magnetic encoder were swapped, placing each in the position meant for the other. Both sensors on the right deck had to be manually wired to the board as a result. One slider had also been accidentally routed to a ground pin rather than an analog input, which was fixed by reassigning it to the analog pin originally used for the main volume knob. These modifications required hand-soldering additional wires directly to the PCB.

A further issue arose with the Trill Bar. It had been connected to the secondary I2C bus on the PCB for layout convenience, but the Trill library only supports the default I2C bus. This was resolved by modifying the library source code directly to support the second bus, though keeping the Trill Bar on the default bus from the start would have avoided this entirely.

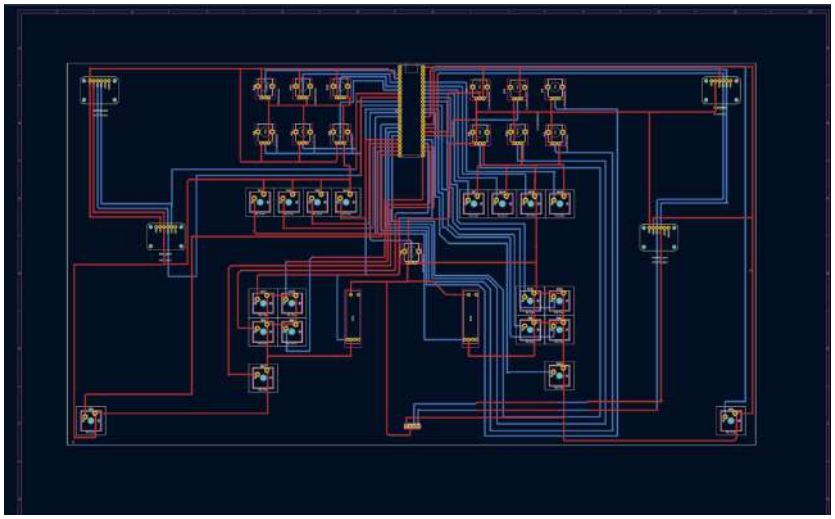

Figure 2: PCB layout in KiCad.

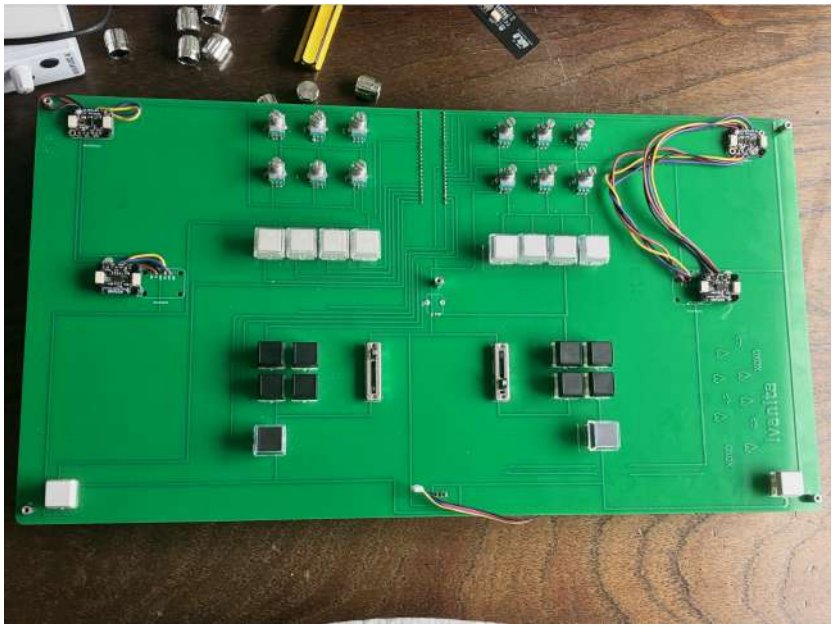

Figure 3: PCB with manual wiring visible.

The enclosure face was laser cut from recycled PLA sheet. Based on prior work on laser cutting PLA [Lara et al., 2022], a starting point of 10% speed, 60% power, and 5000 Hz frequency was used, with power increased to 65% to account for the greater thickness of the material. During cutting, a misalignment shifted the design and exhausted part of the sheet, and the button holes were found to be undersized after the first cut. These issues were resolved through careful realignment against the laser bed corner and a second pass to enlarge the button holes. A section that was accidentally cut through was repaired using PLA welding.

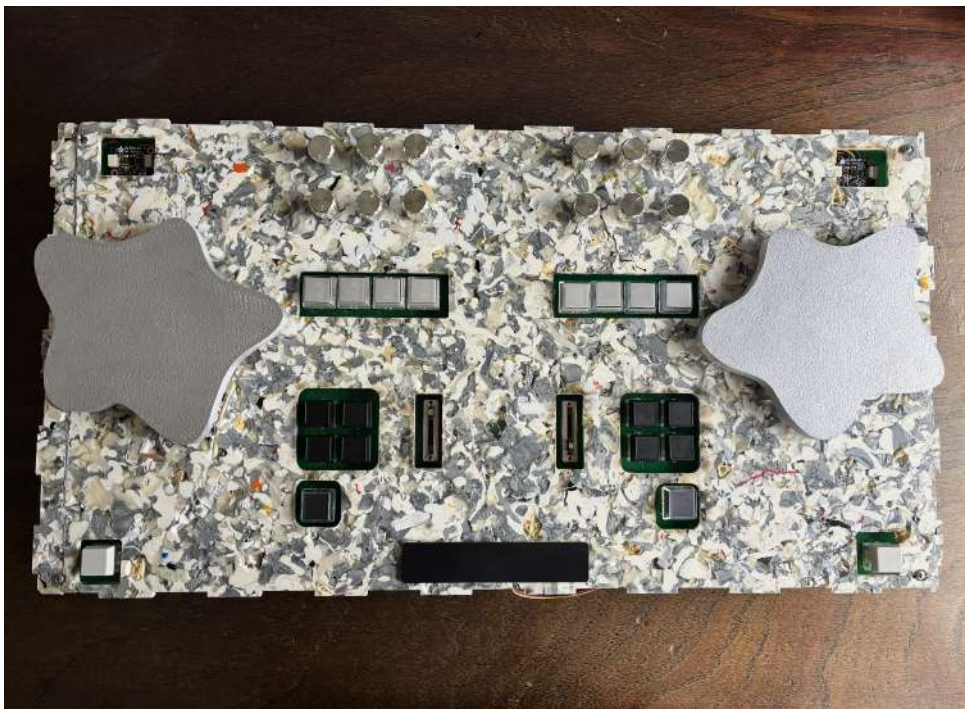

Figure 4: Fully assembled DJ controller with custom enclosure.

The DJ platters were designed in Fusion 360 and 3D printed using a Bambu Lab printer. Each platter mounts on a lazy Susan bearing seated above a circular aperture in the enclosure, with an AS5600 encoder positioned beneath it. A magnet embedded in the underside of each platter allows the encoder to detect rotation. During assembly, one platter sat higher than the other because the lazy Susan hardware was catching on itself during rotation. The platter was hot glued

at a slightly raised position to sit above the enclosure surface, but this introduced a height difference between the two platters that caused significant noise in one of the encoders. After attempting to resolve this through software filtering and radial magnet repositioning without success, it became clear that the vertical distance between the magnet and the sensor surface was the critical variable. Moving the magnet closer to the AS5600 eliminated the noise completely.

Figure 5: Platter with hot glued lazy Susan and magnet.

3.3.2 Control Layout and MIDI Assigment

The controller firmware was written in Arduino C++ using the Teensyduino environment. Four libraries handle sensor communication and input processing: `StevesAwesomeButton` manages button debouncing using the Teensy's internal pull-up resistors, `ResponsiveAnalogRead` applies smoothing to potentiometer readings to eliminate noise, the AS5600 library handles magnetic encoder communication over I2C, and the Adafruit VL53L0X library manages distance sensor polling. The Trill library required modification to its source code to support the secondary I2C bus as described in Section 3.3.1.

All controls transmit on MIDI channel 1. The twenty buttons send `note_on` and `note_off` messages, with `note_on` at velocity 127 on press and `note_off` at velocity 0 on release. The fourteen potentiometers send Control Change mes-

sages, with raw analog values mapped from 0–1023 to 0–127. The encoders, distance sensors, and Trill Bar also transmit as Control Change messages. Table 1 summarizes the full MIDI assignment for all controls and Figure 6 shows a diagram with each control mapping.

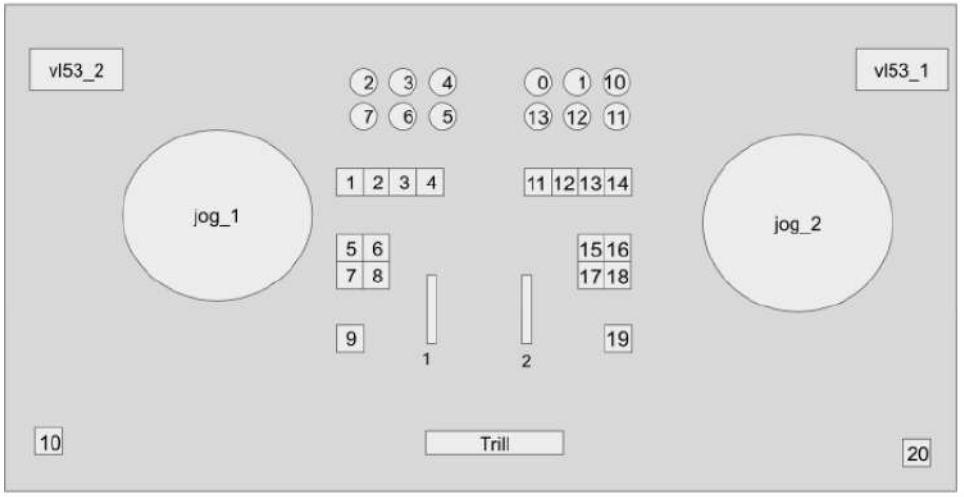

Figure 6: Annotated control layout of the custom DJ controller.

Control	Type	MIDI Message	CC / Note
Jog 1 (left platter)	AS5600 encoder	CC	CC9
Jog 2 (right platter)	AS5600 encoder	CC	CC11
VL53 1 (top right)	Distance sensor	CC	CC13
VL53 2 (top left)	Distance sensor	CC	CC10
Trill Bar	Capacitive touch	CC	CC12
Knob L-Top-2	Potentiometer	CC	CC22
Knob L-Top-3	Potentiometer	CC	CC23
Knob L-Top-4	Potentiometer	CC	CC24
Knob L-Bot-5	Potentiometer	CC	CC25
Knob L-Bot-6	Potentiometer	CC	CC26
Knob L-Bot-7	Potentiometer	CC	CC27
Knob R-Top-0	Potentiometer	CC	CC20
Knob R-Top-1	Potentiometer	CC	CC21

Control	Type	MIDI Message	CC / Note
Knob R-Top-10	Potentiometer	CC	CC30
Knob R-Bot-11	Potentiometer	CC	CC31
Knob R-Bot-12	Potentiometer	CC	CC32
Knob R-Bot-13	Potentiometer	CC	CC33
Slider 1	Potentiometer	CC	CC28
Slider 2	Potentiometer	CC	CC29
Button 1	Tactile button	Note	73
Button 2	Tactile button	Note	72
Button 3	Tactile button	Note	71
Button 4	Tactile button	Note	70
Button 5	Tactile button	Note	65
Button 6	Tactile button	Note	68
Button 7	Tactile button	Note	66
Button 8	Tactile button	Note	69
Button 9	Tactile button	Note	67
Button 10	Tactile button	Note	74
Button 11	Tactile button	Note	58
Button 12	Tactile button	Note	57
Button 13	Tactile button	Note	56
Button 14	Tactile button	Note	55
Button 15	Tactile button	Note	59
Button 16	Tactile button	Note	60
Button 17	Tactile button	Note	61
Button 18	Tactile button	Note	62
Button 19	Tactile button	Note	63
Button 20	Tactile button	Note	64

Table 1: MIDI controller mapping.

The AS5600 encoders output a raw 12-bit angle value between 0 and 4095 representing the full 360-degree rotation of the platter. Rather than sending this raw value directly as MIDI, the firmware uses an accumulator to convert continuous rotation into directional speed messages. On each loop iteration, the difference between the current and previous angle reading is calculated. Because the encoder wraps around at 4096, the firmware checks whether the difference exceeds 2048 and corrects for wrap-around accordingly, ensuring that crossing the zero point in either direction is detected correctly.

Small differences below the jitter threshold are ignored entirely. Jog 1 uses a jitter threshold of 5 and jog 2 uses a threshold of 10, reflecting the greater mechanical noise in jog 2 caused by the height difference described in Section 3.3.1. Differences above the threshold are added to a running accumulator. When the accumulator crosses a motion threshold of 15, the firmware determines whether the motion is slow or fast by comparing the accumulator value to a speed threshold of 50. Forward rotation sends CC value 64 for slow and 65 for fast. Reverse rotation sends 62 for slow and 61 for fast. The accumulator is reset to zero after each MIDI message is sent. This design means the encoder only sends MIDI when meaningful motion is detected, and distinguishes between slow and fast gestures without requiring precise timing measurements.

The VL53L0X sensors poll continuously and return distance readings in millimeters. Raw readings are first filtered to remove invalid values, specifically any reading above 4000mm or equal to 8190, which is the error code the sensor returns when no object is detected within range. Valid readings are then passed through an exponential smoothing filter with an alpha value of 0.25, meaning each new reading contributes 25% to the smoothed value while the previous smoothed value contributes 75%. This reduces sudden spikes in the output while still allowing the value to respond to real hand movement. The smoothed value is mapped from a physical range of 32mm to 300mm to a MIDI range of 0 to 127, and a new MIDI message is only sent if at least 10 milliseconds have passed since the last transmission and the value has changed by at least 1.

The Trill Bar is polled every 50 milliseconds. When a touch is detected, the raw position value, which ranges from 0 to 3194 across the length of the bar, is mapped to a MIDI CC value between 0 and 127. A MIDI message is only sent when the position value has changed since the last reading. When no touch is detected, no message is sent and the last transmitted value is held.

The potentiometers are read on every loop iteration using the `ResponsiveAnalogRead` library, which applies internal smoothing to the 10-bit analog reading and only flags a change when the value has shifted meaningfully. When a change is detected, the value is mapped from the raw range of 0 to 1023 to a MIDI CC range of 0 to 127 and transmitted. Sending only on change rather than on every loop iteration reduces unnecessary MIDI traffic and keeps the output stable when knobs are not being touched.

3.4 Audio-Domain Style Transfer Exploration

Early research conversations raised the question of whether audio-domain style transfer might offer promising possibilities for the DJ controller, prompting an exploratory phase in which several models were investigated. The real-time requirement was at times deprioritized in favor of simply achieving sonically satisfying output, but even this more modest goal was not consistently met, ultimately reinforcing the value of returning to the symbolic MIDI domain.

3.4.1 Google Magenta Real Time

Magenta RealTime [Team et al., 2025] was the first audio-domain model explored. It is an open-weights, low-latency generative music model capable of producing continuous audio streams conditioned on text or audio prompts. As an exploratory experiment, Magenta RT was fine-tuned on a curated 88-minute playlist of Indian music to evaluate whether playlist-based fine-tuning could shift its stylistic output, the full details of which are documented in [RaschChinchilla, 2025]. Informal listening comparisons indicated improved stylistic alignment, with the fine-tuned model producing pitch contours suggestive of raga structures and rhythmic patterns resembling tala cycles. While the model produced musically convincing results and was capable of real-time generation, it did not retain the melodic identity of the source material when conditioned on input audio, instead generating new content consistent with the target style. This mismatch between generative behavior and the requirement for content-preserving transformation led to the decision to explore alternative approaches.

3.4.2 Timbre Transfer Approaches

Following the limitations identified with Magenta RealTime, the next phase centered on timbre transfer as a framework for content-preserving style transformation, with the goal of changing the timbral character of one piece of music to match another while keeping the original melody and structure intact.

AFTER [Demerlé et al., 2024] propose AFTER (Audio Feature Transfer), a diffusion-based model that separates musical structure and timbre into two distinct representation spaces, enabling one-shot timbre transfer and MIDI-to-audio generation. AFTER was selected due to its conceptual alignment with the content-preserving transformation requirement. However, pretrained checkpoints were

trained on narrow instrumental domains, inference proved inconsistent with new input audio, and outputs were frequently artifact-prone and did not reliably preserve the structure of the input.

RAVE [Caillon and Esling, 2021] introduce RAVE (Realtime Audio Variational autoEncoder), a VAE-based model capable of high-quality audio synthesis in real-time, which supports timbre transfer through its encode-decode pipeline. Experiments were conducted using pretrained weights, which likely limited output quality. The resulting audio did not preserve the original musical content sufficiently, and the sonic output appeared too abstract for meaningful integration into a DJ performance context.

DDSP [Engel et al., 2020] propose DDSP (Differentiable Digital Signal Processing), a framework combining neural networks with classical signal processing to give explicit control over pitch, loudness, and timbre. Pretrained weights trained on wind instruments were used alongside a stem separation pipeline to simplify the input. The output was cleaner than AFTER and RAVE but still unconvincing, with poor sound quality, an overly synthetic character, and additional artifacts introduced by stem separation. Although DDSP offers more direct parameter control, this did not translate into meaningful expressive control in practice, making it unsuitable for live performance use.

3.5 MusicVAE Integration

3.5.1 Model Selection and Rationale

MusicVAE [Roberts et al., 2018] is a hierarchical variational autoencoder that operates in the symbolic MIDI domain. It encodes input melodies into a continuous latent space and supports interpolation between any two latent representations, producing musically coherent intermediate sequences between two melodic endpoints. Attribute vectors can be computed as directions in the latent space corresponding to musical attributes, and adding or subtracting these vectors from existing latent codes produces targeted changes to the decoded sequence. The `mel_small` checkpoint was implemented as it was already integrated into the Melody Mixer demo used as the starting point for the browser-based interface. Unlike the hierarchical decoder used in the original MusicVAE paper, `mel_small` uses a flat decoder operating on 2-bar melodic sequences quantized at 16th note resolution and encoded into a 256-dimensional latent space.

In contrast to the audio-domain models explored previously, Melody Mixer provided an accessible and well-documented open-source starting point that could be directly modified and extended, with a clear implementation tutorial available for reference.[10] While the model is limited to monophonic, quantized sequences with no explicit representation of harmony, its pretrained encoder and decoder are directly accessible, making attribute vector arithmetic straightforward to implement without retraining. These properties suited the real-time performance context better than any of the audio-domain alternatives explored.

3.5.2 Browser-Based Interface Design

Melody Mixer allows the user to select two preset melodies and generate interpolations between them, running in the browser using the Magenta.js library to load MusicVAE model weights locally, Tone.js for audio scheduling and playback, and p5.js for visual rendering. The first modification replaced the Melody Mixer's drag-to-extend interpolation with a fixed 11-step interpolation knob. The adapted interface calls MusicVAE's interpolate function to generate 11 evenly spaced points along the spherical interpolation path between two encoded melodies, storing the decoded sequences in memory. Each time the performer activates the loop function, a fresh interpolation is computed between the current chunks loaded on each deck. At performance time, the interpolation knob selects which sequence to play back with no further model inference required. This established the core interaction model of the system: the performer navigates precomputed interpolation by turning a knob, and the interface plays back the corresponding sequence in real time.

A set of real-time performance parameters was then added to expand expressive possibilities beyond interpolation alone. Octave shifting and pitch shifting were implemented as semitone offsets applied to each note at playback without altering the underlying stored sequences. Note length was added as a percentage scaling of each note's quantized duration. Swing was implemented by applying a timing delay to notes landing on odd-numbered 16th note steps, creating the characteristic uneven rhythm where every other beat lands slightly late.

[10] See Blankensmith, https://medium.com/@torinblankensmith/melody-mixer-using-deeplearn-js-to-mix-melodies-in-the-browser-8ad5b42b4d0b

3.5.3 MIDI File Pipeline

To use personal MIDI files as input, a Python script was written using the `pretty_midi` library to extract notes, quantize their timings to 16th note resolution, and output them as JavaScript objects matching the format expected by the interface. Since MusicVAE's `mel_small` checkpoint operates on 2-bar sequences of exactly 32 steps, each song was divided into sequential 32-step chunks. A monophonization step was applied to ensure only one note occupied each time step. An issue arose related to tempo: slicing each song into chunks required using each song's own original tempo to calculate correct bar boundaries, while playback during interpolation required both songs to share the same tempo. The solution was to slice each song using its own tempo to ensure correct bar alignment, then normalize playback to a shared tempo during interpolation. MIDI files were sourced primarily from piano tutorial channels on YouTube, where many creators provide MIDI files directly in the video description.

Four songs were selected as performance material, chosen to represent a range of melodic characters, tempos, and densities. Table 2 summarizes their key musical properties, including average note density and average melodic interval as measured across all chunks extracted from each song.

Song	BPM	Key	Chunks	Avg Note Density
Avril 14th – Aphex Twin	78	A♭ Major	19	4.917
Galore – Oklou	125	F♯ Major	37	4.973
Berghain – Rosalía	125	A Minor	8	8.562
Veridis Quo – Daft Punk	107	A Minor	16	4.031

Table 2: Musical properties of songs used in the system.

3.5.4 Controller Mapping

The browser-based interface was redesigned to resemble a two-deck DJ software layout, with each deck independently loading a different song. The interpolation knob was replaced by a crossfader positioned between the two decks, mapping the familiar spatial logic of DJ mixing onto the MusicVAE interpolation path. The AS5600 encoder platters were mapped to scrub controls, allowing the performer to manually step through the current sequence by rotating the platter. The VL53L0X distance sensors were mapped to tempo control, allowing BPM adjustment through hand proximity without physical contact. Each deck includes a

volume slider controlling MIDI velocity, chunk navigation buttons, a loop button that locks playback and triggers interpolation, a reverse button, four hot cue buttons, and an independent play button. A bass mode button on each deck switches the performance parameter knobs to control the bass layer independently. Finally, two attribute vector knobs were added per deck for density and interval control, operating directly within the MusicVAE latent space by selecting from precomputed variants at performance time.

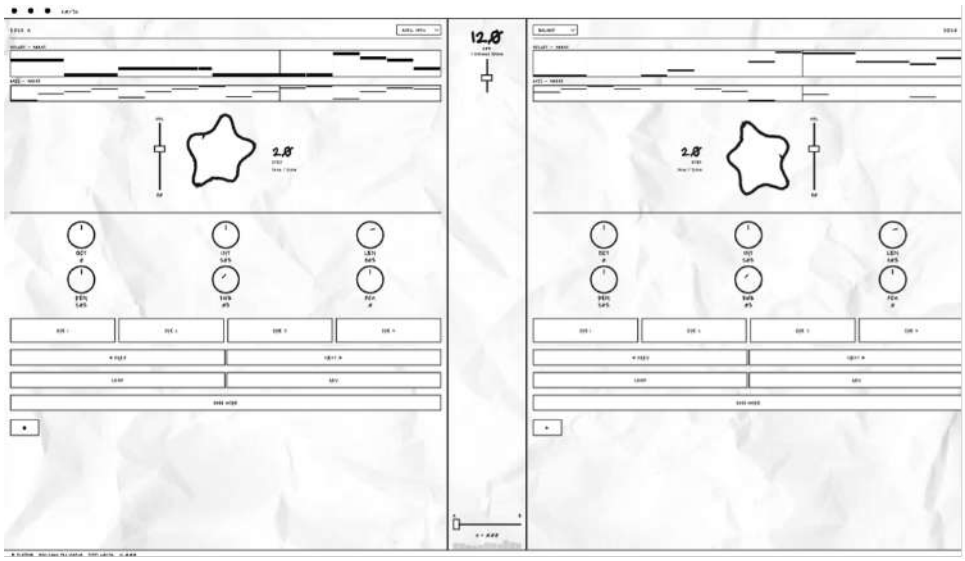

Figure 7: Browser-based interface.

3.5.5 MIDI Output and Disklavier Integration

MIDI sequences are transmitted in real time via the Web MIDI API to a MiniFuse 4 audio interface, which provides a 5-pin DIN MIDI output. This output is connected directly to the MIDI input of a Yamaha Disklavier acoustic piano. All notes are sent on MIDI channel 1. The Disklavier receives the incoming MIDI messages and actuates its internal mechanism to produce acoustic note output, providing a physical acoustic realization of the interpolated melodic sequences generated by MusicVAE.

3.6 Latent Space Interpolation

MusicVAE encodes each input melody using a two-layer bidirectional LSTM encoder [Roberts et al., 2018]. The encoder reads the melody both forwards and backwards, and concatenates the final hidden states from both directions to produce h_T:

$$h_T = [\overrightarrow{h_T}, \overleftarrow{h_T}]$$

This bidirectional context is then passed through two fully connected layers to produce the parameters of the latent distribution:

$$\mu = W_{h\mu} \cdot h_T + b_\mu$$
$$\sigma = \log(\exp(W_{h\sigma} \cdot h_T + b_\sigma) + 1)$$

During training, the latent vector z is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

Given two input sequences A and B encoded as z_A and z_B, interpolation between them is performed in latent space. While Roberts et al. [Roberts et al., 2018] describe using spherical linear interpolation on the basis that linear interpolation tends to pass through low-density regions of the prior in high-dimensional Gaussian spaces, inspection of the Magenta.js source code reveals that linear interpolation is used in practice:

$$c_\alpha = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in \{0.0, 0.1, \ldots, 1.0\}$$

A comment in the source code acknowledges this discrepancy with a note to implement spherical interpolation in a future update. It should also be noted that while [Roberts et al., 2018] describe a hierarchical decoder with a conductor RNN producing per-bar embeddings, the `mel_small` checkpoint used in this implementation uses a flat LSTM decoder operating directly on 2-bar sequences, without the conductor hierarchy.

The system generates 11 evenly spaced steps along this interpolation path, corresponding to α values of 0.0 through 1.0. Step 0 returns Deck A's melody and step 10 returns Deck B's melody, with steps 1 through 9 representing intermediate sequences generated by the model. The crossfader position maps directly

to selecting which of these 11 precomputed sequences to play back, with no further model inference required at performance time. A monophonization step is applied to each input sequence before interpolation to ensure clean single-note input, since MusicVAE's `mel_small` checkpoint is optimized for monophonic melodic input.

3.6.1 Latent Space Attribute Vectors

Beyond the crossfader interpolation, two performance parameters operate directly within the MusicVAE latent space: density and interval. These are implemented as attribute vectors, directions in the 256-dimensional latent space that correspond to musically meaningful attributes, computed offline and applied at performance time through precomputed variant selection.

For this implementation, both attribute vectors were derived from the Lakh MIDI Dataset (`lmd_matched` subset). A corpus of 12,701 monophonic melody chunks was extracted from 500 MIDI files using a custom extraction pipeline, with each chunk quantized to 32 steps at 4 steps per quarter note to match the `mel_small` checkpoint's training format. Tracks that were predominantly monophonic were selected, then strict monophony was enforced on the extracted chunks so that any slight overlaps from imprecise MIDI notation were trimmed. Each extracted chunk also had to contain at least two notes to be considered a candidate for encoding into the latent space. Each chunk was then encoded using MusicVAE's encoder, producing a 256-dimensional latent vector.

Each chunk was scored by its target attribute. Note density was measured as the number of notes per 2-bar sequence, and average melodic interval was measured as the mean absolute pitch difference in semitones between consecutive notes sorted by onset time. The attribute vector was computed as the difference between the mean latent vector of the top quartile and the mean latent vector of the bottom quartile:

$$v = \mu(z \mid \text{high attribute}) - \mu(z \mid \text{low attribute})$$

The L2 norm of the resulting vector,

$$\|v\| = \sqrt{\sum_{d=1}^{256} v_d^2}$$

measures the geometric magnitude of the attribute direction in latent space.

The resulting density vector had a norm of 3.766 and the interval vector a norm of 1.298, indicating that note density constitutes a stronger and more geometrically distinct direction in the latent space than melodic interval. To apply the attribute vector at performance time, each song chunk is pre-encoded into its latent representation z. The chunk is then shifted along the attribute direction at 11 evenly spaced levels:

$$z' = z + \alpha \cdot v, \quad \alpha \in \{-2,\ -1.6,\ -1.2,\ -0.8,\ -0.4,\ 0,\ 0.4,\ 0.8,\ 1.2,\ 1.6,\ 2\}$$

$$z' = z + \alpha \cdot v, \quad \alpha \in \{-15,\ -12,\ -9,\ -6,\ -3,\ 0,\ 3,\ 6,\ 9,\ 12,\ 15\}$$

Each shifted latent vector is decoded back into a note sequence using MusicVAE's decoder and stored as a precomputed variant. This preprocessing was performed offline for all chunks across four songs that the interface has stored, and produces 11 variants per chunk stored in JSON format and loaded by the browser at startup. At performance time, turning the density or interval knob selects which precomputed variant to play, with no real-time neural network inference required. For density, $\alpha = -2$ produces the sparsest melodic variant and $\alpha = +2$ the densest. For interval, $\alpha = -15$ produces the most stepwise melodic motion and $\alpha = +15$ the most intervalically leaping.

Since both attribute variants are precomputed independently, combining them multiplicatively would require precomputing $11 \times 11 = 121$ variants per chunk, which would make the JSON files significantly large for browser loading. Instead, the two attributes operate in a priority system at performance time: whichever knob deviates further from its center position (50 out of 100) takes exclusive control of the melody selection for that deck. Formally, if $|\alpha_{\text{den}} - 0.5| \geq |\alpha_{\text{int}} - 0.5|$, the density variant is selected; otherwise the interval variant is selected. When both knobs are at center, the original unmodified chunk plays. This design ensures that the performer always has one active attribute transformation per deck.

3.7 Evaluation

Evaluation focuses on hardware reliability, control responsiveness, and the integration of the MusicVAE interpolation system with the custom controller, with emphasis on how well the system supports controllable interaction during performance. Hardware evaluation was conducted through a systematic validation

protocol covering all sensor and control types on the controller, documented in Section 4.1. Each audio-domain model was tested against sonic quality and real-time controllability, with a Pure Data prototype used to assess the timbre transfer pipeline in practice, documented in Section 4.2. The final MusicVAE system was evaluated in terms of interpolation behavior, interface responsiveness, and controller-to-model integration, documented in Section 4.3. Performance testing sessions were used to assess control clarity, comfort, latency, and musical flow.

3.8 Ethical Considerations

This project does not involve human participants, personal data collection, or user studies, and therefore does not require institutional review board approval. All performance testing was conducted by the researcher. MIDI files used as performance material were sourced from publicly shared files provided by piano tutorial creators on YouTube, and their use is strictly for non-commercial academic research. The Lakh MIDI Dataset, used to compute attribute vectors for the latent space controls, is a publicly available research dataset used in accordance with its academic license. Software tools, neural models, and DJ platforms used throughout the research are open source or publicly documented, and the hardware design follows principles of transparency and reproducibility. The generative model functions as a performance tool rather than an autonomous creative system, with all musical decisions remaining in the hands of the performer.

4 Results

4.1 Hardware Validation

A systematic hardware validation protocol was developed and executed to assess the reliability, accuracy, and performance of the custom DIY DJ controller prior to integration with the MusicVAE latent space interpolation system. The validation process comprised of six quantitative tests targeting each category of sensor and control input present on the controller: potentiometers, buttons, encoders, capacitive touch, and time-of-flight distance sensors. Data was collected using a Python logging script interfacing with the Teensy microcontroller via USB-MIDI, using the `mido` library to capture incoming MIDI messages. Through the logging script all raw data was saved to CSV files and analyzed using an analysis script, with figures generated using `matplotlib`. The validation system was designed to confirm that each control meets the functional thresholds required for reliable live performance use, and to point out any hardware anomalies likely occurred by manual wiring modifications described in Section 3.3.1.

4.1.1 Potentiometer Response and Range Validation

MIDI range and response linearity tests were performed for all 14 potentiometer-based controls across a full sweep from minimum to maximum physical position. Thirteen of the fourteen controls achieved full-scale range coverage with observed values of 1 to 126, with the one-unit offset from the theoretical 0–127 range attributable to the physical limitations of the potentiometers at their endpoints combined with the integer rounding behavior of the Arduino `map()` function. Zero directional reversals and no crosstalk were detected across all thirteen controls. Slider 2 (CC29) was the exception, reaching only 2 to 113 and producing large non-linear value jumps rather than smooth sequential increments, as illustrated in Figure 8. This instability is attributable to the unstable physical connection resulting from the manual wiring required after the PCB footprint error described in Section 3.3.1. Despite this hardware limitation, Slider 2 remained sufficiently usable during performance for coarse parameter control.

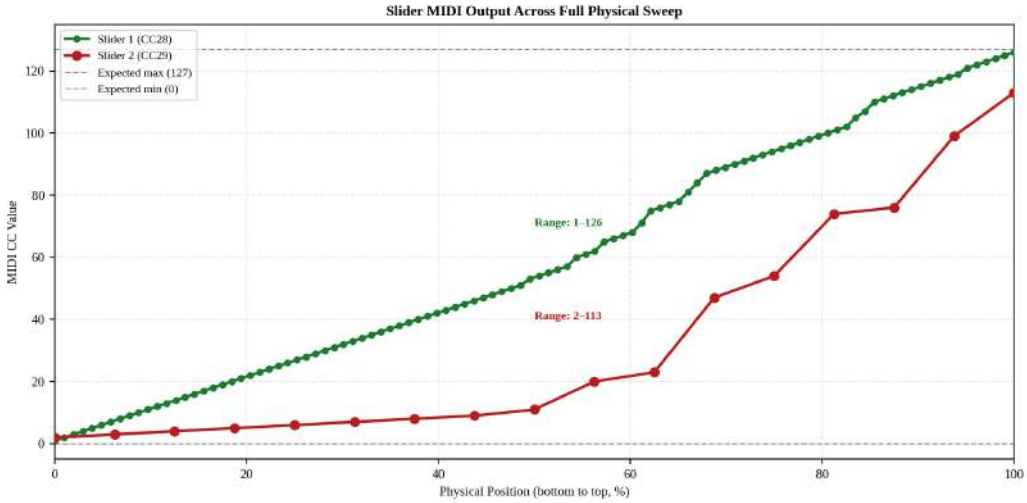

Figure 8: MIDI output comparison between Slider 1 and Slider 2

4.1.2 Button Reliability

Button reliability was assessed across all 20 buttons. Each button was pressed 50 times by the researcher, and the total number of registered `note_on` events was logged. Double triggers were defined as two consecutive `note_on` events on the same MIDI note within a 50-millisecond window. Results are illustrated in Figure 9. 16 of the 20 buttons achieved a detection rate of 100%. No double triggers were observed on any button. The remaining four buttons showed minor discrepancies, with detection rates ranging from 92% to 102%. Since all testing was performed manually by the researcher while keeping a count, these small differences are likely due to human counting error rather than hardware failure. The absence of double triggers across all 20 buttons supports this conclusion.

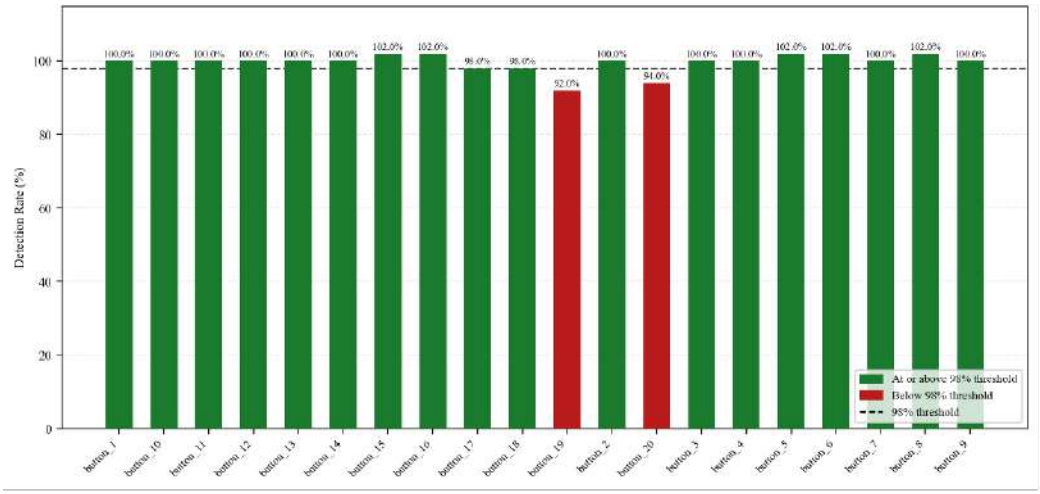

Figure 9: MIDI output comparison between Slider 1 and Slider 2

4.1.3 MIDI Transmission Consistency

MIDI transmission timing was assessed across four control types by measuring inter-event timing: the elapsed time in milliseconds between consecutive MIDI messages arriving at the Python logging script, which captures the consistency of the transmission pipeline and identifies any irregular delays or dropouts. Each control was triggered continuously by the researcher while incoming messages were logged. Knobs and sliders were swept repeatedly, the Trill Bar was slid back and forth, the distance sensor was approached and retreated from, and the platters were rotated. Results are summarized in Table 3 and illustrated in Figure 10. The knob and platter controls both produced a median inter-event time of 6.16ms, reflecting the shared firmware loop timing rather than coincidence, as neither control has a fixed polling interval and both transmit at the natural rate of the Teensy's main loop when actively moved. The distance sensor produced a median of 40.35ms and the Trill Bar a median of 50.26ms, both consistent with their respective fixed firmware polling intervals of 10ms and 50ms, which regulate transmission rate independently of gesture speed. The large maximum values observed for the knob (1609ms) and platter (2121ms) correspond to deliberate pauses between gestures during data collection rather than transmission delays and are not representative of continuous use. All four control types demonstrated consistent and stable transmission behavior with no evidence of systematic dropout or delay.

Table 3: MIDI transmission latency by control type

Control Type	N Samples	Median (ms)	Mean (ms)	Min (ms)	Max (ms)
Knob	4816	6.16	13.41	0.07	1609.46
Crossfader	791	50.26	55.85	47.65	410.76
Distance Sensor	1066	40.35	47.91	3.79	440.76
Platter	5387	6.16	9.38	0.06	2121.95

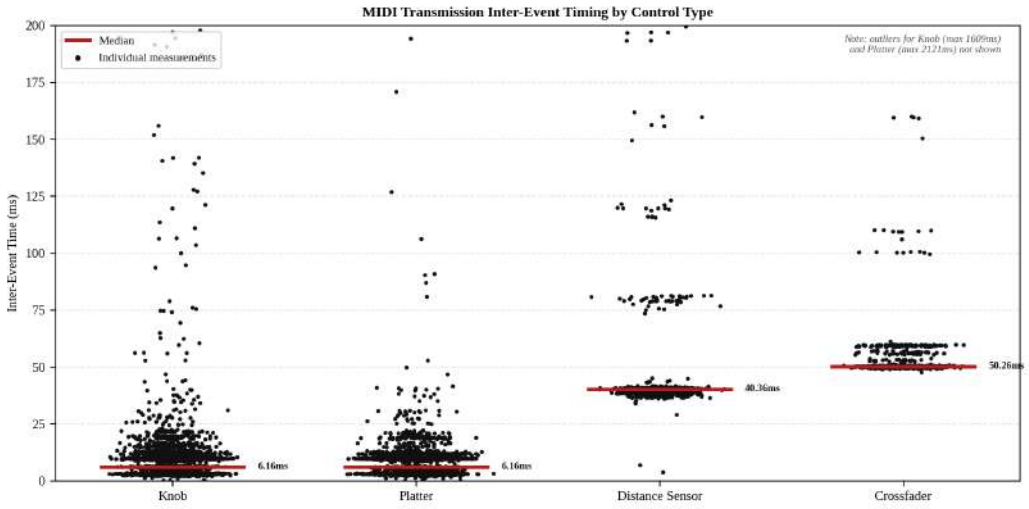

Figure 10: MIDI inter-event timing by control type, capped at 200ms. Knob and platter exhibit outliers up to 2121ms corresponding to deliberate pauses during data collection.

4.1.4 AS5600 Encoder Performance

The two AS5600 magnetic rotary encoders assigned to CC9 (`jog_1`, left platter) and CC11 (`jog_2`, right platter) were evaluated across four sub-tests. Sub-test A confirmed that neither encoder produced spurious messages while held still for ten seconds, confirming that the `JITTER1` and `JITTER2` firmware thresholds effectively suppress noise-induced false triggers. Sub-test B assessed speed scaling during slow and fast rotations, with fast-coded values accounting for only 42.7% and 43.5% of messages for `jog_1` and `jog_2` respectively during fast rotation, attributable to the `FAST_SPEED=50` threshold being set deliberately high to prevent false triggers during slow gestures. Sub-test C confirmed 100% directional accuracy for both encoders across ten full rotations in each direction. Figures for sub-tests A through C are not shown as results are summarized above.

Sub-test D assessed gesture repeatability by performing ten scratch gestures of consistent intended size and speed on each platter and logging how many MIDI messages each gesture produced. If the encoder is consistent, the same gesture should produce roughly the same number of messages each time, which matters for performance because the scrubbing feature advances through the 32-step MIDI sequence in proportion to how many messages the platter sends. `jog_1` produced an average of 446 messages per gesture with a coefficient of variation of 22%, and `jog_2` produced an average of 449 messages with a variation of 35%, though excluding the first trial which produced an unusually high count of 840 messages likely due to an accidentally larger gesture, the variation drops to 19%, comparable to `jog_1`. Both encoders produced broadly consistent output, with the observed variation attributable to the natural imprecision of performing the same gesture by hand ten times. For the intended use case this level of consistency is sufficient, as the system requires that spinning more produces more messages and spinning less produces fewer.

Table 4: AS5600 encoder sub-test results summary for `jog_1` and `jog_2`

Sub-test	Metric	`jog_1`	`jog_2`
A: Dead Zone	Spurious messages (10s rest)	0	0
B: Speed Scaling	Fast-coded values during fast rotation	42.7%	43.5%
C: Directional Consistency	Accuracy (CW and CCW)	100%	100%

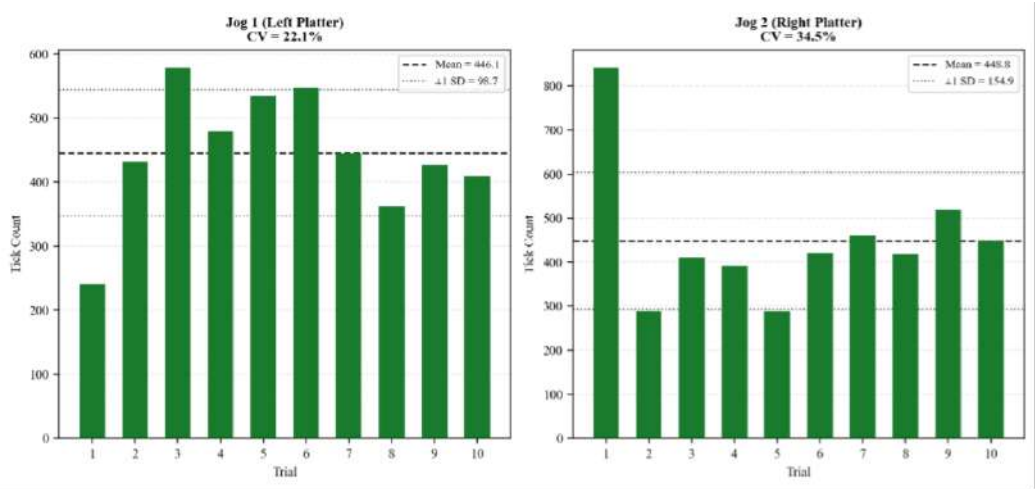

Figure 11: MIDI message count per scratch gesture across ten trials for `jog_1` and `jog_2`.

4.1.5 Trill Bar Position Accuracy

The Trill Bar capacitive touch sensor, assigned to CC12, was evaluated for positional accuracy by measuring its MIDI output at five target positions along its length: 0%, 25%, 50%, 75%, and 100%. Each position was held for three seconds while values were logged, and the median observed value was compared against the expected MIDI value for that position. The test was conducted across three separate runs to assess consistency across trials. Due to the physical contact area of a human finger, it is not possible to activate a single precise MIDI value on the Trill Bar. Positional accuracy is therefore evaluated not against a single precise target value but against an acceptable tolerance range. The full MIDI range of 0 to 127 is divided into five equal zones corresponding to the five target positions, each zone spanning approximately 25.4 MIDI values. This zone width defines the acceptable tolerance for each target position, as a performer's finger naturally covers approximately one fifth of the bar's length when placed at any given position. A sensor reading is therefore considered accurate if the observed median falls within ± 25.4 MIDI values of the expected target. Results from the primary test run are summarized in Table 5 and illustrated in Figure 12.

All five target positions produced median values within the acceptable tolerance range. The 0% position yielded a median of 1 against an expected value of 0, the 25% position yielded a median of 30.5 against an expected value of 32, the 50% position yielded a median of 60 against an expected value of 64, the 75%

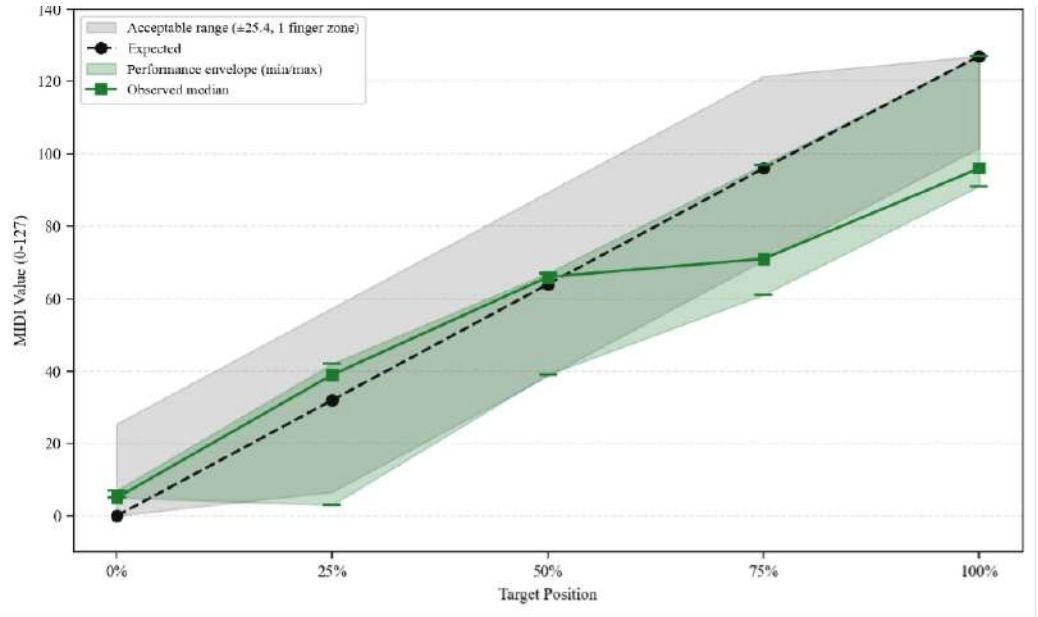

Figure 12: Trill Bar Positional Response

position yielded a median of 84 against an expected value of 96, and the 100% position yielded a median of 124 against an expected value of 127. The observed performance envelope remained within or close to the acceptable tolerance band at all five positions, confirming that natural finger placement variation does not push values outside the intended zone.

Table 5: Trill Bar positional accuracy results. Tolerance defined as ±25.4 MIDI values.

Target Position	Expected	Median	Min	Max	Within Tolerance
0%	0	1	0	2	Yes
25%	32	30.5	0	36	Yes
50%	64	60	28	68	Yes
75%	96	84	67	85	Yes
100%	127	124	84	125	Yes

4.1.6 VL53L0X Distance Sensor Linearity

The two VL53L0X time-of-flight distance sensors, assigned to CC10 (`vl53_1`) and CC13 (`vl53_2`), were evaluated for linearity and noise behavior across four target distances: 5cm, 10cm, 20cm, and 30cm. Each distance was held for five seconds while MIDI CC values were logged. Both sensors demonstrated clear monotonic increases in median output value as distance increased, confirming that the distance-to-MIDI mapping is directionally consistent. For `vl53_1`, median values were 22 at 5cm, 44 at 10cm, 84 at 20cm, and 110.5 at 30cm. For `vl53_2`, median values were 25 at 5cm, 55 at 10cm, 91 at 20cm, and 117 at 30cm. Results are summarized in table 6 and illustrated in Figures 13 and 14.

Both sensors exhibited noise, defined as the range between minimum and maximum observed values at each distance. The large noise ranges, particularly for `vl53_1` at shorter distances with a range of 118 MIDI values at 5cm, are partly attributable to transient spikes at the beginning of each measurement window before the hand was fully settled at the target distance. `vl53_2` exhibited more contained noise overall, with ranges of 69 at 5cm, 45 at 10cm, 42 at 20cm, and 69 at 30cm. The remaining noise is consistent with the known behavior of VL53L0X sensors in open-air environments without optical baffling, and is suppressed during performance by the exponential smoothing filter implemented in the Teensy firmware. The median values nonetheless confirm that both sensors reliably discriminate between the four tested distances and produce a usable gestural range for expressive control.

Table 6: VL53L0X distance sensor median output and noise range by target distance

	`vl53_1`		`vl53_2`	
Distance	Median	Noise Range	Median	Noise Range
5cm	22	118	25	69
10cm	44	108	55	45
20cm	84	85	91	42
30cm	110.5	76	117	69

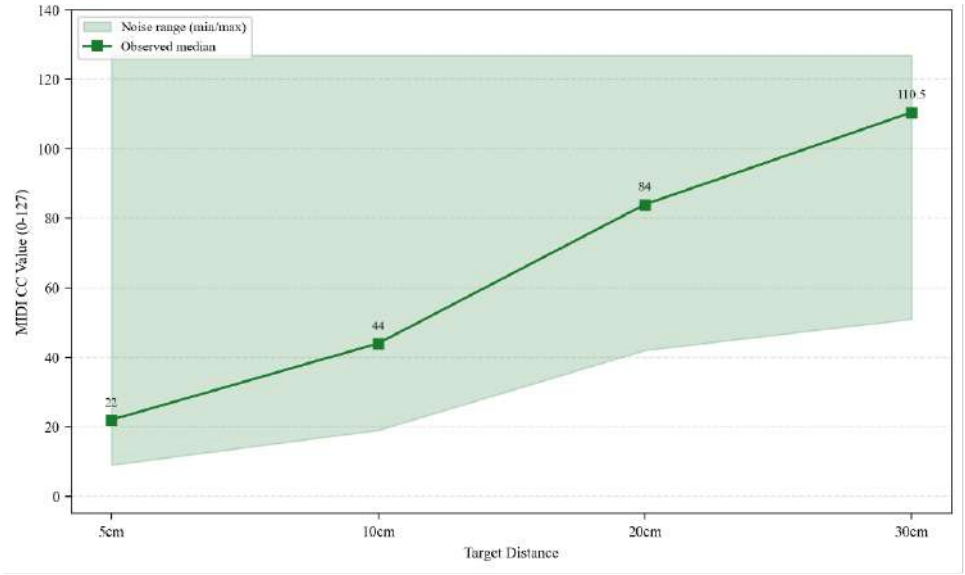

Figure 13: VL53L0X `v153_1` Distance Sensor Linearity and Noise Range

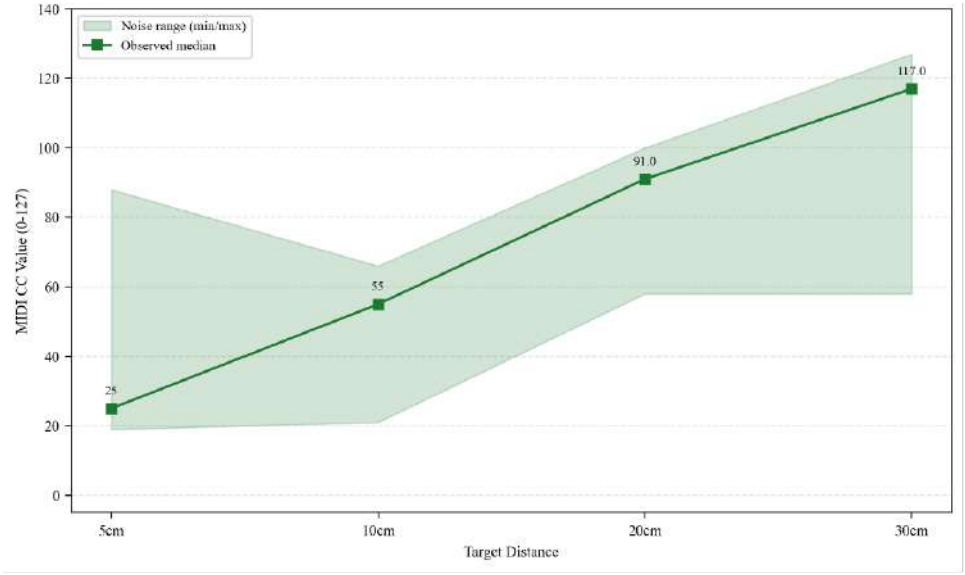

Figure 14: VL53L0X `v153_2` Distance Sensor Linearity and Noise Range

4.2 Audio Domain Exploration Findings

4.2.1 Pure Data Performance Prototype

To evaluate the timbre transfer pipeline as a performance tool rather than purely an offline processing system, a prototype interface was developed in Pure Data. Since real-time processing was not feasible within the pipeline, timbre transfer was performed offline in advance and the resulting audio was loaded into the patch for playback. The patch provided independent control over four stem channels (drums, bass, vocals, and other), alongside four timbre selectors, allowing the performer to switch between processed versions of each stem during playback. This made it possible to evaluate the performance workflow in practice rather than in model testing alone. The experience confirmed that even in the best-case offline scenario the sonic quality of the timbre-transferred stems was not convincing enough for live performance, and the static nature of pre-processed audio felt disconnected from the expressive potential of the hardware controller.

This outcome was consistent with the broader findings of the model selection process. None of the models explored satisfied the dual requirements of content preservation and real-time controllability, reflecting the current state of the field rather than the limitations of any individual model. Stem separation introduced its own artifacts that compounded with those of timbre transfer, degrading output at multiple stages of the pipeline. Reproducing published results also proved consistently harder than expected, and the gap between documented performance and behavior in a novel deployment context was significant in every case. Switching to the symbolic MIDI domain and MusicVAE was a practical decision based on the available tools. MusicVAE offered stable interpolation, direct latent space control, and real-time performance compatibility out of the box, without requiring custom model training, which none of the audio-domain models could provide.

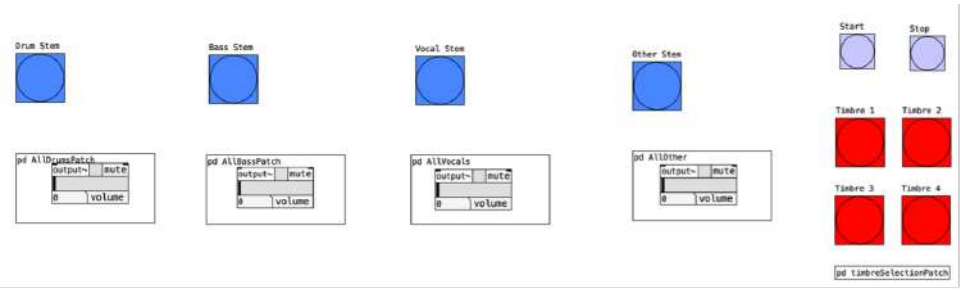

Figure 15: Pure Data timbre transfer prototype patch.

4.3 Quantitative Validation of the Latent Space System

This section evaluates two deep learning components of the Neural DJ system: the MusicVAE crossfader interpolation and the note density and average interval attribute vectors. Crossfader smoothness is assessed by measuring Hamming distance across the 11-step interpolation path, following the methodology of [Roberts et al., 2018]. Attribute vector effectiveness is assessed by measuring whether traversal along each vector produces directional change in the corresponding musical attribute, quantified using monotonicity rate and Spearman rank correlation

4.3.1 Interpolation Validation

Following the evaluation methodology of [Roberts et al., 2018], Hamming distance was computed between each of the 11 interpolated sequences and the original deck A sequence, measuring the proportion of the 32 time steps that differ as the crossfader advances from A to B. A Bernoulli baseline was constructed by sampling each time step independently:

$$p(x_t = b_t) = \alpha, \quad p(x_t = a_t) = 1 - \alpha$$

whose expected Hamming distance increases linearly with α. All six song pair combinations were evaluated across five chunk pairs each, producing 30 interpolations total. As shown in Figure 16, all six pairs rose more steeply than their respective Bernoulli baselines, reaching high Hamming distances by $\alpha = 0.4$ to 0.5. This reflects the decisive melodic transitions characteristic of the flat `mel_small` decoder, and monotonic behavior was maintained across all pairs, confirming directionally consistent melodic morphing.

Five of the six song pairs achieved a mean Hamming distance of 0.00 at $\alpha = 0$, indicating near-perfect reconstruction. The Véridis Quo $\times$ Berghain pair was the exception, with a mean intercept of 0.28, visible as the elevated brown curve in Figure 16. The per-chunk breakdown in Appendix B reveals that this pair also exhibited the highest variance across chunk pairs, with individual curves ranging from 0.00 to 0.69 at $\alpha = 0$. To contextualize these differences, the total latent path length between each song pair was computed by summing the Euclidean distances between consecutive interpolated steps in latent space. As shown in Table 7, Avril 14th $\times$ Galore had the shortest mean path length (29.908), consistent with their similar note densities (4.917 and 4.973 notes per bar respectively), while all three Berghain pairs had the longest paths (45.7 to 46.1), reflecting its substantially higher note density (8.562).

Song Pair	**Mean Path Length**	**Std**
Avril 14th $\times$ Galore	29.908	4.492
Galore $\times$ Véridis Quo	33.240	5.184
Avril 14th $\times$ Véridis Quo	41.878	3.775
Véridis Quo $\times$ Berghain	45.712	11.333
Avril 14th $\times$ Berghain	45.845	12.747
Galore $\times$ Berghain	46.084	9.121

Table 7: Mean total latent path length between song pairs across five chunk pairs.

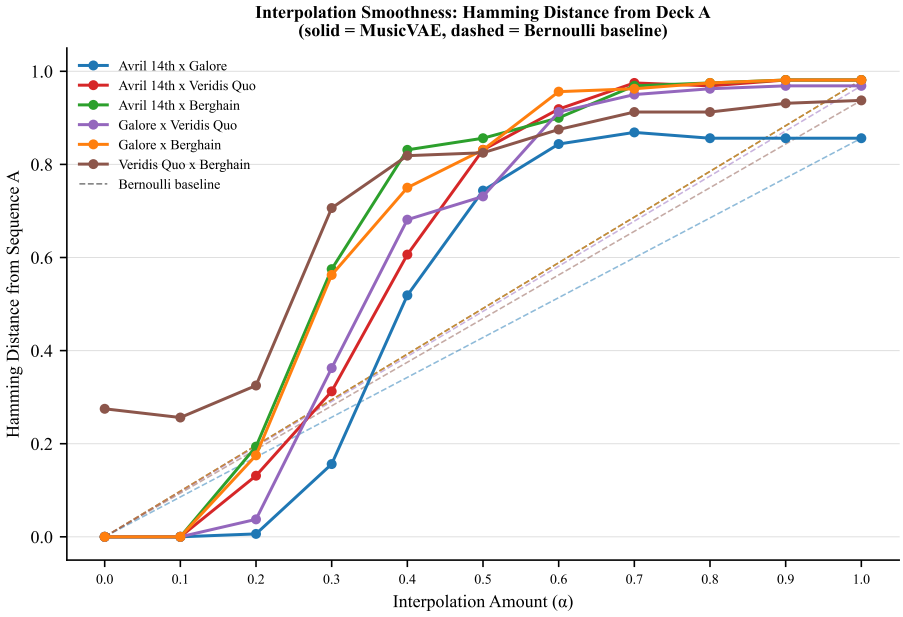

Figure 16: Interpolation validation linear chart.

4.3.2 Attribute Vector Validation

To evaluate the effectiveness of the latent space attribute vectors, two quantitative metrics were computed for each attribute: monotonicity rate and Spearman rank correlation. Monotonicity rate measures the proportion of chunks for which the target attribute increases consistently across the 11 precomputed variants, defined as at least 7 of 10 consecutive variant transitions being non-decreasing. Spearman rank correlation measures the strength of the overall relationship between variant index and attribute value across all 11 steps, where a value of 1.0 indicates a perfect monotone increase and 0.0 indicates no relationship. Both metrics were computed across all chunks of all four personal MIDI files: Avril 14th, Galore, Véridis Quo, and Berghain, yielding a total of 79 chunks analyzed per attribute.

Note Density The density attribute vector produced near-perfect results across all four songs. The monotonicity rate was 100% for every song, meaning that for every chunk in the dataset, turning the DEN knob from position 0 to position 10 consistently produced a strictly increasing number of notes per bar. The mean

Spearman correlation across all 79 chunks was $\rho = 0.989$ with a standard deviation of 0.011, indicating an extremely strong and consistent relationship between knob position and note density.

The mean note density values across the 11 variants ranged from 0.08 notes per bar at variant 0 to 16.00 notes per bar at variant 10, spanning the full range from near silence to maximum melodic density. This gradient was consistent across all four songs despite their different melodic characters and tempos, demonstrating that the density vector represents a robust and generalizable direction in MusicVAE's latent space rather than a song-specific artifact. These results confirm that the note density attribute vector reliably controls the number of notes produced by the decoder across the full range of the knob, and that this control is musically meaningful rather than random variation.

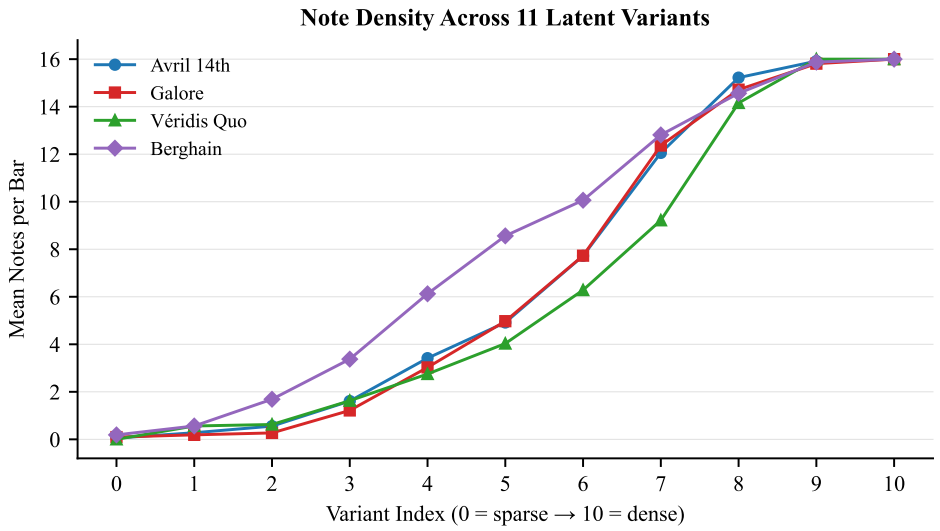

Figure 17: Note density across latent variants linear chart.

Average Melodic Interval The interval attribute vector produced weaker but directionally consistent results. Monotonicity rates varied across songs, from 37.5% for Berghain to 75.0% for Véridis Quo, with an overall mean Spearman correlation of $\rho = 0.362$ and a standard deviation of 0.531. While this indicates a positive trend across the variant axis, the high variance suggests that the interval vector does not produce consistent step-to-step increases for the majority of

chunks. The weaker performance of the interval vector is consistent with its lower norm of 1.298 compared to the density vector norm of 3.766. A smaller norm indicates that the vector represents a less geometrically distinct direction in the latent space, requiring a larger scalar multiplier to produce perceptually meaningful variation. For this reason the shift range for the interval vector was set to $\alpha \in \{-15, -12, -9, -6, -3, 0, 3, 6, 9, 12, 15\}$ rather than the ± 2 range used for the density vector. Despite the increased range, the interval vector produces audible changes primarily at the extremes of the knob rather than across the full 11-step gradient.

This result is theoretically coherent. Note density is a global and structurally salient property of a melody that the decoder must represent explicitly in order to reconstruct sequences accurately. Average melodic interval, by contrast, is entangled with pitch content, phrasing, and harmonic context, making it a less cleanly disentangled axis in the latent space. The contrast between the two vectors illustrates a key property of MusicVAE's learned representation: not all musically meaningful attributes are equally separable in latent space, and the degree of disentanglement depends on how central the attribute is to the model's reconstruction task.

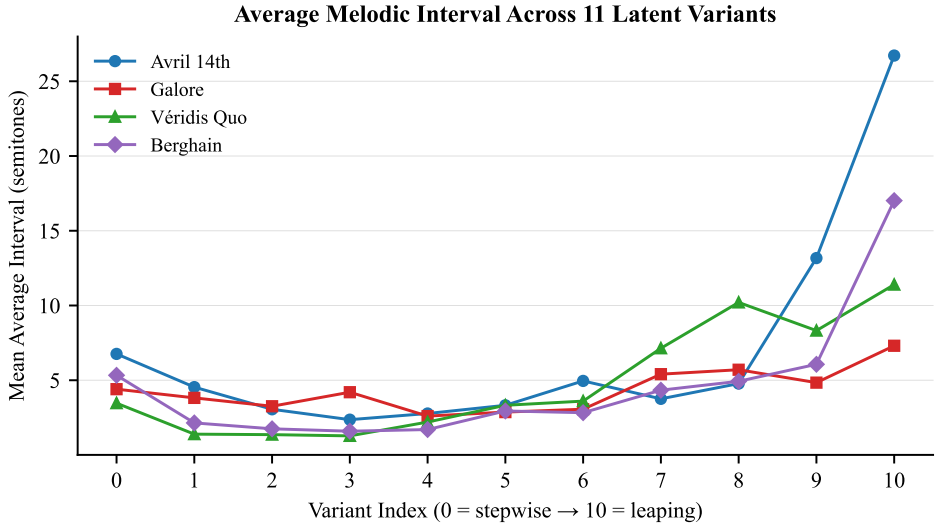

Figure 18: Average melodic interval across latent variants linear chart.

Table 8: Spearman rank correlation coefficients for note density and average melodic interval attribute vectors across all four songs

Song	Note Density ρ	Average Interval ρ
Avril 14th	0.993	0.334
Galore	0.988	0.227
Véridis Quo	0.996	0.743
Berghain	0.973	0.290
Overall	0.989 ± 0.011	0.362 ± 0.531

4.4 Qualitative Integrated System Evaluation

4.4.1 Latent Space Interaction Behavior and Musical Output

The MusicVAE interpolation system produced musically coherent output across the 11-step interpolation path. Informal listening tests were performed across all four song pairs, with transitions feeling smooth and gradual across all pairs regardless of musical similarity. A notable pattern emerged in how musical dimensions changed along the path: pitch content shifted earlier and more noticeably than rhythm, with rhythmic changes becoming more pronounced around steps 4 and 5. This suggests pitch is more strongly encoded along the interpolation axis than rhythm in MusicVAE's latent space. Song pairs that were more similar in note density felt more musically compatible during interpolation, producing transitions that felt more natural. The note density knob performed smoothly across its full range. The knob center corresponds to the original sequence density, with leftward rotation decreasing density and rightward rotation increasing it. The decrease direction did not feel performatively meaningful, and the jump from center to the first step up felt somewhat abrupt. The average melodic interval knob was less effective as a performance control. Changes felt random rather than directional, and turning the knob caused notes to drop from the generated sequence. However, it could still be useful for intentional drastic interval changes rather than subtle continuous control.

4.4.2 Controller Behavior with Browser Interface

Integration testing confirmed that all mapped controller inputs performed as expected in conjunction with the browser interface. The Trill Bar successfully controlled interpolation position, selecting the correct interpolation step in response

to finger position along the sensor. The distance sensor reliably adjusted tempo within the mapped BPM range. Although the sensor exhibited some sensitivity to small hand movements, consistent with the noise results reported in Section 4.1.6, this did not produce erratic tempo jumps and BPM remained stable relative to hand position. The performance parameter knobs responded correctly to physical input, with changes to octave, pitch shift, velocity, density, swing, and note length reflected immediately in playback. The AS5600 encoder platters successfully triggered scrubbing when rotated, stepping through the 32-step sequence in proportion to platter movement.

During fast platter scrubbing, the playhead moved faster than notes could be triggered, causing some steps to be skipped in the Disklavier. Slider 2's reduced range did not produce significant perceptual differences during integration testing. While resolution felt slightly more abrupt compared to Slider 1 under close attention, overall behavior was consistent.

4.4.3 Browser Based Interface Reflection

The browser-based interface was adapted from a Magenta real-time demo, which introduced certain constraints while also guiding the reimplementation process. Overall, the knobs and buttons functioned reliably and as intended. However, the page often required manual refreshing whenever a song was paused or reached its end, which could reduce usability during performance. One of the most notable usability issues arises during looping after interpolation between songs. Both songs must be looped before interpolation. Otherwise, when the interpolation finishes and control shifts to the second track, turning off the loop of the previous song can cause the playback to jump to the next section unexpectedly. To avoid this, both decks need to be set up correctly with looping before starting interpolation.

These issues stem primarily from software design decisions rather than conceptual limitations of the model. They also raise questions regarding the necessity and clarity of having separate loop controls per deck. At an earlier stage of the design, an alternative approach was considered in which one deck controlled melody and the other controlled bass. However, the final implementation adopted a more conventional dual-deck DJ layout, where each deck corresponds to a separate track while sharing transport controls. In hindsight, a more unified or abstracted control scheme may have improved usability and reduced ambiguity.

4.4.4 Disklavier Integration and MIDI Output

MIDI output from the browser interface was successfully routed to the Disklavier via the MiniFuse 4 and a 5-pin DIN MIDI cable. The piano accurately reproduced the pitch, velocity, and timing of generated sequences in real time, providing an acoustic realization of the interpolated melodic output. A small delay was observed between the browser interface piano roll display and the physical actuation of the Disklavier hammers, which is expected given the mechanical nature of the instrument. Outside of the scrubbing limitation described in Section 4.3.3, the Disklavier integration functioned reliably throughout testing and produced acoustically clear output consistent with the MIDI data generated by the system.

5 Discussion

This section directly addresses the four research questions that motivated this practice-based research thesis. Each sub-section is organized around one research question, drawing on the hardware validation results, latent space system validation results, iterative model selection findings, and integrated system evaluation presented in section 4.

5.1 Expressive Control of the DIY Controller

The first research question asks what the technical and ergonomic limitations of using a custom DJ controller to navigate MusicVAE latent spaces are.

The validation results show a positive response overall, as every hardware component performed as expected with the exception of one slider. A key reason the system performed as reliably as it did was the decision to precompute both the interpolation steps and the attribute vector variants offline. Had the system been performing MusicVAE inference in real time, parameter changes during performance would have risked interrupting or corrupting the generation process. Slider 2's reduced smoothness was the most noticeable hardware limitation, but its effect was minimal in practice. Because it controlled MIDI velocity, which maps to loudness, and human loudness perception is logarithmic and does not resolve the full 127 discrete levels available in the MIDI velocity range, the reduced resolution was not perceptually significant during performance. The distance sensor introduced a more meaningful limitation in practice. Because the sensor continuously maps hand height to BPM in real time, placing the hand near the sensor immediately changes the tempo to whatever position the hand lands at, with no ability to approach gradually. Removing the hand also required sliding it out horizontally rather than lifting it vertically, since any upward motion caused the BPM to jump to its maximum value. An improvement would be to implement a locking mechanism where the current BPM is held until the performer deliberately engages the sensor again, freeing the hand between tempo adjustments.

Several interaction patterns produced expressive outcomes during performance. Combining the density knob with octave shift or pitch shift was particularly effective, as changes to the generative content and changes to its transposition compounded in ways that felt musical and intentional. Note length also interacted meaningfully with swing: shorter note lengths made the swing effect more perceptible, while longer note lengths masked it. The interval knob was the least expressive of the performance parameters because it produced changes that felt

too drastic, though this could be useful in performances where an abrupt melodic shift is intentional. A more fundamental limitation was that density and interval could not be used simultaneously due to their precomputed nature, restricting the performer to one attribute transformation at a time. Ergonomically, the knobs felt slightly too close together during performance, and the Trill Bar's placement made it occasionally susceptible to accidental triggering when reaching for neighboring controls. A more effective placement for the Trill Bar would be along the top of the controller, aligned horizontally with the distance sensors rather than positioned at the bottom.

These observations point toward future work on controller ergonomics, simultaneous attribute control, and deeper exploration of what expressive dimensions MusicVAE's latent space has to offer beyond the two attribute vectors implemented here.

5.2 Feasibility for Individual Builders

The second research question asks what the process of building a custom DMI for deep learning interaction reveals about the current feasibility of this kind of system for individual instrument builders working with open-source tools.

The build process confirmed that this kind of system is feasible for an individual builder, but that feasibility depends heavily on the quality of available documentation and the ability to troubleshoot across multiple domains at once. The most decisive factor in resolving hardware problems was not technical expertise alone but whether a component had enough community support and clear documentation to fall back on. For example, not all components worked out as planned. The VL53L4CX [11] distance sensor was selected first but abandoned after its sparse documentation made debugging infeasible, and the VL53L0X was brought in as a replacement and proved far easier to integrate. The Trill Bar is a sensitive component that broke during prototyping and required replacement; careful handling is essential of this specific sensor. The AS5600 encoder required the magnet to be positioned very precisely relative to the sensor surface, something that only became apparent after encountering significant noise in the output.

Fabrication decisions also had significant cost and reliability implications. Designing a custom PCB reduced wiring complexity and improved reliability, but it is not inexpensive and introduced its own risk in the form of layout errors that

[11] Adafruit VL53L4CX Time of Flight Distance Sensor: `https://www.adafruit.com/product/5425`

required manual rewiring after fabrication. Using hand-soldered point-to-point wiring throughout would have been cheaper but would have increased the likelihood of connection faults during performance. Building with recycled materials proved to be a difficult process of the project. Sourcing used keycaps was very difficult, and working with reclaimed PLA required significant iteration to achieve usable results. While sustainability is a worthwhile goal, the difficulty of sourcing recycled components and working with reclaimed materials reflects a broader gap in the field. This is a gap worth addressing as the maker and instrument building communities continue to grow.

Overall, building a custom DMI for deep learning interaction is accessible to an individual builder working with open-source tools, but the process demands sustained tolerance for trial and error at every stage.

5.3 Limitations of Deep Learning Models for Live Performance

The third research question asks what the limitations of current audio-domain and MIDI-domain deep learning music models are for controllable and content-preserving real-time performance.

Beyond the quality limitations of the audio domain models explored, the model exploration process surfaced a deeper question about what kind of style transfer is actually appropriate for a given live performance context. As discussed in the literature review, prior systems that embed RAVE within interactive interfaces tend to treat the latent space as open territory for exploration, where the abstract and unpredictable nature of the output is part of the artistic intent. [Shaheed and Wang, 2024] and [Scurto and Postel, 2023] both design systems where physical gesture navigates a RAVE latent space without a fixed destination, and the instability of the output is not a limitation but a feature. This is a coherent and artistically legitimate approach, but it reflects a different set of performance goals than those of this thesis. The system developed here was not designed to produce completely unfamiliar sounds. It was designed to transform familiar ones. The central performance argument of this thesis is that audiences should be able to recognize the source material and follow its transformation in real time, which means the output cannot be fully abstract. When so much of the source material is lost to artifacts, the audience loses the thread of the original song and with it the ability to engage with the transformation as a musical event rather than a purely sonic one. This distinction points to a difference not just between models but between two broader approaches to style transfer in live performance: timbral transfer, which transforms the sonic identity of a sound, and compositional transfer, which trans-

forms the melodic and structural content while preserving enough familiarity for the listener to follow.

This reorientation also brought its own set of limitations. MusicVAE does not support real-time inference within the constraints of a browser-based performance interface, requiring all songs to be sliced into fixed-length chunks and all interpolated sequences and attribute vector variants to be precomputed offline in advance. The quantization process required to force notes onto a fixed temporal grid. This removes subtle timing variations that contribute to each composition's distinctive feel, reducing aspects of its expressive character. The model also requires a monophonization step that discards harmonic information present in the original MIDI files. The interval attribute vector performing less reliably than the density vector, suggests that not all musical attributes are equally accessible through arithmetic operations on MusicVAE's latent space.

The central limitation of current models for content-preserving real-time performance is that timbral and compositional transformations are not yet jointly controllable. Audio-domain models can modify timbre but often degrade melodic structure, while MIDI-domain models preserve composition but discard timbre. Combining both in a real-time, performance-oriented system remains an open challenge.

5.4 MusicVAE Latent Space as a Performance Control System

The fourth research question asks how latent space interpolation and attribute vectors in MusicVAE function as control parameters for musical features such as density, contour, and key, and where these controls break down.

The four songs differ in tempo, key, melodic contour, and note density, and these differences shaped how interpolation behaved in practice. Note density had the largest influence on latent distance: Berghain's higher density produced the steepest Hamming curves and longest latent paths across all pairs involving it, while Avril 14th and Galore, with nearly identical densities, produced the most gradual curve, leveling off around 0.85 and failing to reach full separation even at $\alpha = 1.0$.

Galore has a rising melodic contour while Véridis Quo has a descending one, and the interpolated output between them combined both into material that rose and then fell within a single chunk, suggesting that MusicVAE encodes melodic contour in a perceptually legible way. The Avril 14th and Galore pair highlighted a limitation of the Hamming metric: despite a gradual note-level transition, the shift between A flat major and F sharp major created a noticeable tonal boundary

around alpha 0.5. This suggests that key transposition before interpolation would be a useful addition to the system, similar to the capability found in conventional DJ controllers. The Véridis Quo and Berghain pair produced a relatively smooth melodic transition, likely because both songs are in A minor. A subtle density shift was noticeable around alpha 0.3, consistent with the steeper rise in the Hamming curve at that point. This effect may be partly due to structural differences between the two songs. Véridis Quo contains sections with very low note density, whereas Berghain is consistently sparse, which may cause a sharper transition in density during interpolation. This observation may also be influenced by the fact that melodic interpolation in this pair was relatively smooth, which shifts perceptual attention toward changes in rhythm.

The note density attribute vector produced perceptually consistent and controllable results across all four songs. The interval vector was less effective as a continuous control. Turning the knob up caused the melody to collapse toward a single repeated pitch, and at its maximum notes were skipped entirely. This happens because pushing the vector far enough to produce an audible change moves the latent representation into territory the decoder struggles to reconstruct, so instead of generating wider leaps between notes it produces incoherent output.

Overall, latent space interpolation and attribute vectors in MusicVAE provide partially reliable control over musical features such as density, contour, and key, but can break down under tonal mismatches, as well as when attribute manipulation pushes the decoder beyond its reconstruction capacity.

6 Conclusions

This thesis demonstrates that latent space DJing is technically feasible and musically expressive. By navigating through the space between compositions rather than switching between them, a performer can generate material that preserves enough of the original melody for an audience to recognize it while introducing combinations that are entirely new but share characteristics with both source songs. This balance between the familiar and the unknown is central to what makes a DJ performance compelling, and MusicVAE's latent space offers a principled way to achieve it. The system also demonstrates that building this kind of instrument is feasible for an individual researcher working with open source tools, though it demands sustained troubleshooting across hardware, firmware, and software simultaneously. The custom controller was validated across all sensor categories and communicated reliably with the browser based interface, confirming that a fully integrated system of this kind can be built and performed outside of a commercial setting. Sustainability proved harder to pursue in practice than in principle, reflecting a real gap in the infrastructure available to individual builders worth addressing as the maker community continues to grow. Ultimately, this work presents a proof-of-concept for a new category of live instrument. By embedding generative models within a physical DJ controller, this thesis proves that manipulation of compositional style transfer is a viable and expressive approach to live performance.

6.1 Future Work

Several directions for future work emerge naturally from the findings and limitations of this thesis. Within the existing system, the most immediate extensions would be switching to the hierachical decoder, adding key transposition control, computing attribute vectors from a curated performance playlist rather than the Lakh dataset, and expanding the song library available for live use. At the model level, future symbolic music models with support for polyphonic sequences and unquantized timing would unlock a substantially wider range of musical material. Stronger attribute disentanglement in future models could also make knob-based control more precise and predictable. Because this thesis focused primarily on establishing technical feasibility, future work could also move toward perceptual evaluation: listener studies with DJs and performers assessing which gestural mappings feel most expressive, which attributes are most musically useful, and whether the generative model contributes meaningfully to the performer's sense

of creative control. A particularly exciting longer-term possibility is a system in which compositional style transfer and timbre transfer operate together in a live generative performance context.

Glossary

AFTER (Audio Feature Transfer) A diffusion-based audio model designed for feature-level style conditioning. Explored in this project as a candidate for timbre transfer.

AS5600 A magnetic rotary encoder that detects angular position using a magnet. Used beneath each jog wheel platter to detect rotation direction and speed.

Attribute Vector A direction in a model's latent space corresponding to a musically meaningful attribute. Adding or subtracting it from a latent representation produces targeted changes in the decoded output.

BPM (Beats Per Minute) A measure of musical tempo.

CC (Control Change) A MIDI message type used to transmit continuous parameter values ranging from 0 to 127.

DDSP (Differentiable Digital Signal Processing) A framework combining neural networks with classical signal processing components. Explored as a timbre transfer approach in this project.

DMI (Digital Musical Instrument) An instrument that uses digital hardware and software to generate or transform musical output.

GAN (Generative Adversarial Network) A deep learning model consisting of a generator and discriminator trained in opposition, used for generating realistic data.

GPIO (General Purpose Input/Output) Configurable pins on a microcontroller used for digital input and output.

GRU (Gated Recurrent Unit) A recurrent neural network architecture that uses gating mechanisms to process sequential data efficiently.

I2C (Inter-Integrated Circuit) A two-wire serial communication protocol used to connect microcontrollers to peripheral sensors.

JSON (JavaScript Object Notation) A lightweight data format for storing and exchanging structured data.

Latent Space A compressed, continuous mathematical space learned by a generative model in which similar inputs are encoded as nearby points.

Latent Space Interpolation Navigating smoothly between two encoded points in a model's latent space, producing intermediate outputs that blend characteristics of both inputs.

LSTM (Long Short-Term Memory) A recurrent neural network architecture designed to learn long-range dependencies in sequential data.

MIDI (Musical Instrument Digital Interface) A communication protocol for transmitting musical performance data between electronic instruments and software.

Monophonization The process of ensuring only one note occupies each time step in a MIDI sequence.

MusicVAE A hierarchical variational autoencoder developed by Google Magenta that encodes MIDI melodies into a continuous latent space and supports smooth interpolation between any two encoded sequences.

Neural Style Transfer (NST) A deep learning technique that applies the stylistic characteristics of one input onto another while preserving its underlying structure.

Note Density The number of notes present within a fixed musical time window. Used in this project as an attribute vector for latent space control.

PBR (Practice-Based Research) A research methodology in which knowledge is generated through iterative creative or technical practice rather than observation alone.

PCB (Printed Circuit Board) A board that mechanically supports and electrically connects electronic components using conductive traces.

RAVE A VAE-based real-time audio synthesis model. Explored in this project's audio-domain phase but abandoned due to lack of content preservation.

Stem Separation The decomposition of a mixed audio signal into individual source components such as drums, bass, vocals, and other instruments.

Timbre Transfer The transformation of a sound's tonal quality to match that of a different instrument while preserving its pitch and rhythm.

VAE (Variational Autoencoder) A generative deep learning model that encodes input data into a continuous probabilistic latent space and decodes sampled points back into data.

VL53L0X A time-of-flight distance sensor used in this project to measure hand proximity for non-contact gestural control.

Web MIDI API A browser API that enables web applications to send and receive MIDI data from connected hardware devices.

References

[Brunner et al., 2018] Brunner, G., Konrad, A., Wang, Y., and Wattenhofer, R. (2018). Midi-vae: Modeling dynamics and instrumentation of music with applications to style transfer. *arXiv preprint arXiv:1809.07600*.

[Caillon and Esling, 2021] Caillon, A. and Esling, P. (2021). Rave: A variational autoencoder for fast and high-quality neural audio synthesis.

[Candy, 2006] Candy, L. (2006). Practice based research: A guide. *CCS report*, 1(2):1–19.

[Cífka et al., 2020] Cífka, O., Şimşekli, U., and Richard, G. (2020). Groove2groove: One-shot music style transfer with supervision from synthetic data. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28:2638–2650.

[Dahlstedt, 2014] Dahlstedt, P. (2014). Circle squared and circle keys performing on and with an unstable live algorithm for the disklavier. In *NIME*, pages 114–117.

[Dai et al., 2018] Dai, S., Zhang, Z., and Xia, G. G. (2018). Music style transfer: A position paper. *arXiv preprint arXiv:1803.06841*.

[Demerlé et al., 2024] Demerlé, N., Esling, P., Doras, G., and Genova, D. (2024). Combining audio control and style transfer using latent diffusion. *arXiv preprint arXiv:2408.00196*.

[Engel et al., 2020] Engel, J., Hantrakul, L. H., Gu, C., and Roberts, A. (2020). Ddsp: Differentiable digital signal processing. In *International Conference on Learning Representations*.

[Fiordelmondo et al., 2023] Fiordelmondo, A., Russo, A., Pizzato, M., Zecchinato, L., and Canazza, S. (2023). A multilevel dynamic model for documenting, reactivating and preserving interactive multimedia art. *Frontiers in Signal Processing*, 3:1183294.

[Gatys et al., 2016] Gatys, L. A., Ecker, A. S., and Bethge, M. (2016). Image style transfer using convolutional neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2414–2423.

[Javadi, 2026] Javadi, M. (2026). Timbre transfer in the latent space of descript audio codec: Transforming beatbox performances into acoustic drum kit recordings.

[Katz, 2012] Katz, M. (2012). *Groove music: The art and culture of the hip-hop DJ*. Oxford University Press.

[Kawai et al., 2020] Kawai, L., Esling, P., and Harada, T. (2020). Attributes-aware deep music transformation. In *ISMIR*, pages 670–677.

[Kingma and Welling, 2013] Kingma, D. P. and Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

[Lara et al., 2022] Lara, A., Mahdi, I., Ritmiller, M., and Bill, V. (2022). Recycling 3d print pla: Sheet production for laser cutting. *IJAMM*.

[Leman, 2007] Leman, M. (2007). *Embodied music cognition and mediation technology*. MIT press.

[Leman and Godøy, 2010] Leman, M. and Godøy, R. I. (2010). Why study musical gestures? In *Musical Gestures*, pages 15–23. Routledge.

[Meyer, 1956] Meyer, L. B. (1956). *Emotion and Meaning in Music*. University of Chicago Press.

[Montano, 2010] Montano, E. (2010). 'how do you know he's not playing pacman while he's supposed to be djing?': technology, formats and the digital future of dj culture. *Popular Music*, 29(3):397–416.

[Murray-Browne and Tigas, 2021] Murray-Browne, T. and Tigas, P. (2021). Latent mappings: Generating open-ended expressive mappings using variational autoencoders. In *Proceedings of the International Conference on New Interfaces for Musical Expression*, Shanghai, China.

[Northlich, 2013] Northlich, W. R. (2013). Diy dynamic: Experimental electronic music and the underground in the san francisco bay area.

[Pati and Lerch, 2019] Pati, A. and Lerch, A. (2019). Latent space regularization for explicit control of musical attributes. In *ICML Machine Learning for Music Discovery Workshop (ML4MD), Extended Abstract, Long Beach, CA, USA*.

[Privato et al., 2024] Privato, N., Shepardson, V., Lepri, G., and Magnusson, T. (2024). Stacco: Exploring the embodied perception of latent representations in neural synthesis. In Bin, S. M. A. and Reed, C. N., editors, *Proceedings of the International Conference on New Interfaces for Musical Expression*, pages 424–431, Utrecht, Netherlands.

[RaschChinchilla, 2025] RaschChinchilla, I. (2025). Bringing global music to magenta realtime: Playlist-driven fine-tuning for personalized dj performance. In *Ismir 2025 Hybrid Conference*.

[Rietveld, 2016] Rietveld, H. C. (2016). Authenticity and liveness in digital dj performance. In *Musicians and their Audiences*, pages 139–149. Routledge.

[Roberts et al., 2018] Roberts, A., Engel, J., Raffel, C., Hawthorne, C., and Eck, D. (2018). A hierarchical latent vector model for learning long-term structure in music. In *International conference on machine learning*, pages 4364–4373. PMLR.

[Rydén, 2026] Rydén, O. (2026). Latent space interpolation for music transitions. *E18*.

[Scurto and Postel, 2023] Scurto, H. and Postel, L. (2023). Soundwalking deep latent spaces. In Ortiz, M. and Marquez-Borbon, A., editors, *Proceedings of the International Conference on New Interfaces for Musical Expression*, pages 232–235, Mexico City, Mexico.

[Shaheed and Wang, 2024] Shaheed, N. and Wang, G. (2024). I am sitting in a (latent) room. In Bin, S. M. A. and Reed, C. N., editors, *Proceedings of the International Conference on New Interfaces for Musical Expression*, pages 333–338, Utrecht, Netherlands.

[Sharma and Baghel, 2026] Sharma, B. and Baghel, A. S. (2026). Disentangling style-a supervised factorized vae for controllable polyphonic music generation. In *2026 2nd International Conference on Cognitive Computing in Engineering, Communications, Sciences and Biomedical Health Informatics (IC3ECSBHI)*, pages 1266–1270. IEEE.

[Sonntag et al., 2024] Sonntag, D., Barz, M., and Gouvea, T. (2024). A look under the hood of the interactive deep learning enterprise (no-idle). *arXiv preprint arXiv:2406.19054*.

[Team et al., 2025] Team, L., Caillon, A., McWilliams, B., Tarakajian, C., Simon, I., Manco, I., Engel, J., Constant, N., Li, Y., Denk, T. I., Lalama, A., Agostinelli, A., Huang, C.-Z. A., Manilow, E., Brower, G., Erdogan, H., Lei, H., Rolnick, I., Grishchenko, I., Orsini, M., Kastelic, M., Zuluaga, M., Verzetti, M., Dooley, M., Skopek, O., Ferrer, R., Petridis, S., Borsos, Z., Äaron van den Oord, Eck, D., Collins, E., Baldridge, J., Hume, T., Donahue, C., Han, K., and Roberts, A. (2025). Live music models.

[Vieira and Schiavoni, 2020] Vieira, R. and Schiavoni, F. L. (2020). Fliperama: An affordable arduino based midi controller. In *NIME*, pages 375–379.

[Warren and Çamci, 2022] Warren, N. and Çamci, A. (2022). Latent drummer: A new abstraction for modular sequencers. In McPherson, A. and Frid, E., editors, *Proceedings of the International Conference on New Interfaces for Musical Expression*, Auckland, New Zealand.

[White, 2016] White, T. (2016). Sampling generative networks. *arXiv preprint arXiv:1609.04468*.

[Wyse, 2017] Wyse, L. (2017). Audio spectrogram representations for processing with convolutional neural networks. *arXiv preprint arXiv:1706.09559*.

[Yang et al., 2017] Yang, L.-C., Chou, S.-Y., and Yang, Y.-H. (2017). Midinet: A convolutional generative adversarial network for symbolic-domain music generation. *arXiv preprint arXiv:1703.10847*.

A Schematic

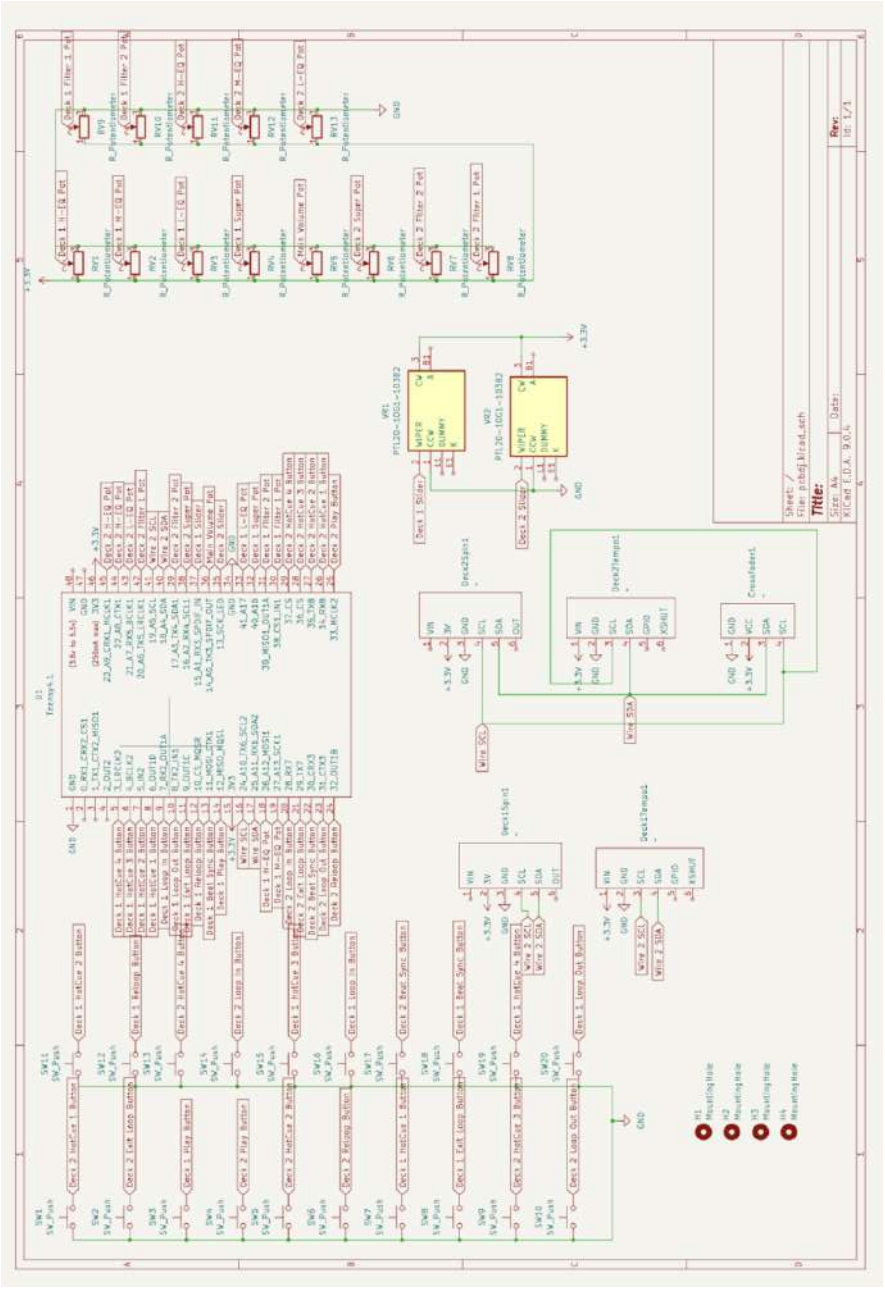

Figure 19: Schematic Diagram of the DIY DJ Controller.

B Interpolation Validation Figures

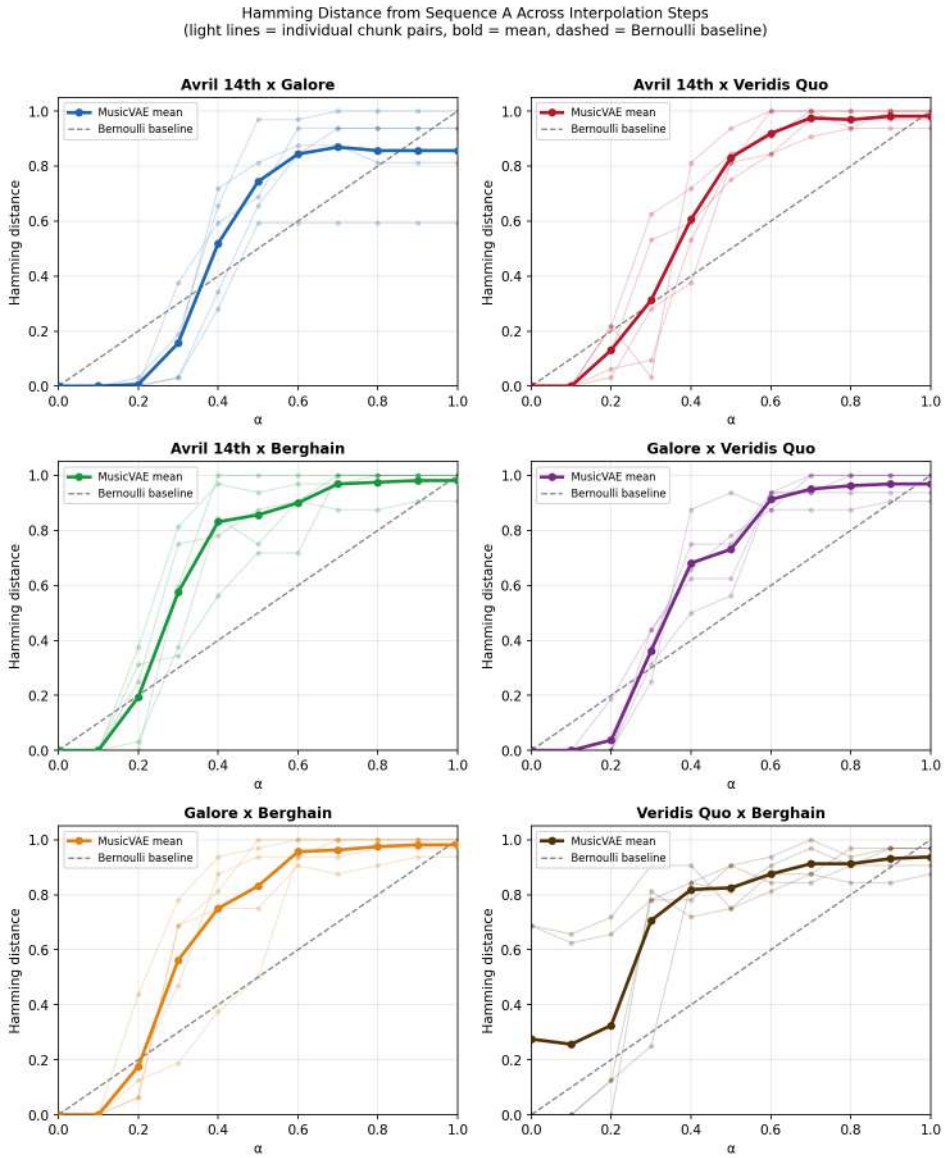

Figure 20: Hamming distance curves for all six song pair combinations across five chunk pairs each. Light lines indicate individual chunk pairs, bold lines indicate the mean, and dashed lines indicate the Bernoulli baseline.